



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática (Ingeniería del Software)

BarGAIN: Sistema inteligente de optimización de la cesta de la compra

**Realizado por
Nicolás Parrilla Geniz**

**Dirigido por
Juan Vicente Gutiérrez Santacreu**

**Departamento
Matemática Aplicada I**

Sevilla, Junio 2026

Resumen

BarGAIN es una plataforma de compra inteligente orientada a optimizar la cesta de la compra del consumidor combinando tres variables de decisión: precio, distancia y tiempo. El proyecto desarrolla una solución full-stack con backend en Django/DRF, base de datos PostgreSQL + PostGIS, cliente móvil en React Native (Expo), y companion web para el portal de comercios.

La propuesta integra cuatro capacidades diferenciales en una única aplicación: (1) comparación multi-tienda de precios, (2) optimización multicriterio de ruta con OR-Tools, (3) digitalización de listas y tickets mediante OCR con Google Cloud Vision, y (4) asistente conversacional basado en LLM (Google Gemini) vía backend proxy.

Desde la perspectiva de ingeniería del software, el trabajo cubre el ciclo completo: análisis de requisitos, diseño arquitectónico, implementación modular, pruebas unitarias/integración/E2E, despliegue en staging y documentación técnica. La arquitectura se organiza en dominios desacoplados (users, products, stores, prices, shopping_lists, optimizer, ocr, assistant, business, notifications), con tareas asíncronas en Celery y Redis para procesos pesados.

Como resultado, se obtiene un prototipo funcional validado en entorno de desarrollo y staging, con cobertura de pruebas en módulos críticos, flujos E2E automatizados para el web companion, y verificaciones manuales en flujos móviles dependientes de GPS/cámara. La memoria documenta además limitaciones, decisiones arquitectónicas y líneas de evolución para convertir BarGAIN en un producto escalable.

Abstract

BarGAIN is an intelligent shopping platform focused on optimizing grocery baskets by combining three decision variables: price, distance, and time. The project delivers a full-stack solution with a Django/DRF backend, PostgreSQL + PostGIS geospatial storage, React Native (Expo) mobile client, and a web companion for small businesses.

The proposal integrates four key capabilities in a single system: (1) multi-store price comparison, (2) multicriteria route optimization with OR-Tools, (3) OCR-based list/ticket digitization using Google Cloud Vision, and (4) a conversational assistant powered by an LLM (Google Gemini) through a backend proxy.

From a software engineering standpoint, the work covers the whole lifecycle: requirements analysis, architecture design, modular implementation, unit/integration/E2E testing, staging deployment, and technical documentation. The architecture is organized into decoupled domains, with asynchronous processing delegated to Celery workers over Redis.

The final outcome is a functional prototype validated through automated and manual tests, with clear traceability of technical decisions, current limitations, and future evolution paths toward production readiness.

Agradecimientos

A mi tutor, Juan Vicente Gutiérrez Santacreu, por su orientación constante, su criterio técnico y su apoyo durante todo el desarrollo del TFG.

A mi familia y personas cercanas, por su paciencia y motivación en las fases de mayor carga de trabajo.

A la comunidad de software libre y a la documentación técnica de las herramientas utilizadas, que han sido clave para avanzar con rigor en el diseño, la implementación y la validación de BarGAIN.

Índice general

Índice general	IV
Índice de cuadros	VIII
Índice de figuras	IX
Acrónimos	XI
1 Introducción y objetivos del proyecto	1
1.1 Contexto y motivación	1
1.2 Problema abordado	1
1.3 Objetivo principal	1
1.4 Objetivos específicos	1
1.5 Alcance del proyecto	2
1.6 Tecnologías clave	2
1.7 Estructura de la memoria	3
2 Análisis de antecedentes y aportación realizada	4
2.1 Contexto del sector	4
2.2 Estado del arte en comparación de precios	4
2.3 Tecnologías relevantes para BarGAIN	4
2.3.1 Geoespacial y rutas	4
2.3.2 Ingesta de datos de precio	5
2.3.3 OCR y asistente	5
2.4 Aportación diferencial de BarGAIN	5
3 Comparación con otras alternativas	6
3.1 Competidores considerados	6
3.2 Matriz funcional resumida	6
3.3 Brechas estratégicas detectadas	7
3.4 Posicionamiento de BarGAIN	7
3.5 Riesgos competitivos	7
4 Análisis temporal y costes de desarrollo	8
4.1 Planificación temporal	8
4.2 Metodología de trabajo y trazabilidad del proceso	8
4.2.1 Planificación viva: el árbol .planning/	8
4.2.2 Trazabilidad de tareas y evidencias	9
4.2.3 Agentes de IA con contrato de contexto explícito	9
4.2.4 Registro de errores y reglas derivadas	9
4.2.5 Valoración del enfoque	9

4.3	Estimación de costes	10
4.4	Riesgos y mitigaciones	10
4.5	Resultado de planificación	10
5	Análisis de requisitos	11
5.1	Actores del sistema	11
5.2	Requisitos de información	11
5.3	Requisitos funcionales	11
5.4	Criterios de aceptación del producto	12
5.5	Requisitos no funcionales	12
5.6	Reglas de negocio	13
5.7	Diagramas de casos de uso	13
5.8	Trazabilidad	13
6	Diseño e Implementación	16
6.1	Arquitectura del Sistema	16
6.1.1	Visión general	16
6.1.2	Patrón de organización interna del backend	16
6.1.3	Comunicación entre componentes	17
6.2	Modelo de Datos	17
6.2.1	Módulo de usuarios	17
6.2.2	Módulo de productos	18
6.2.3	Módulo de tiendas	18
6.2.4	Módulo de precios	18
6.2.5	Módulo del optimizador	19
6.2.6	Índices clave	19
6.2.7	Modelo entidad-relación	19
6.2.8	Diagrama de clases	19
6.3	Diseño de la API REST	19
6.3.1	Estructura general	19
6.3.2	Endpoints principales	21
6.3.3	Autenticación y autorización	21
6.4	Implementación del Backend	22
6.4.1	Módulo de notificaciones	22
6.4.2	Módulo de portal business	22
6.5	El Algoritmo de Optimización de Rutas	22
6.5.1	Formulación del problema	22
6.5.2	Fase 1 — Resolución semántico-difusa de ítems	23
6.5.3	Fase 2 — Consolidación de paradas	23
6.5.4	Fase 3 — Matriz de distancias y tiempos	23
6.5.5	Fase 4 — Ordenación de la ruta con OR-Tools	24
6.5.6	Fase 5 — Persistencia y desambiguación interactiva	24
6.5.7	Diagrama de secuencia del optimizador	24
6.5.8	Evolución respecto al diseño inicial	24
6.5.9	Complejidad y salvaguardas de rendimiento	24
6.6	Módulo de Web Scraping	26
6.6.1	Consideraciones legales y éticas del scraping	26
6.7	Módulo OCR	27
6.7.1	Diagrama de secuencia OCR	27
6.8	Integración del Asistente LLM	27
6.9	Infraestructura y Despliegue	29
6.9.1	Contenedores Docker	29

6.9.2	Pipeline CI/CD	29
6.9.3	Variables de entorno	29
6.10	Diseño de la Interfaz de Usuario	30
6.10.1	Sistema de diseño	30
6.10.2	Pantallas principales	30
6.10.3	Adaptación a uso web (responsive y conveniencias de escritorio)	30
7	Manual de Usuario	32
7.1	Introducción	32
7.2	Despliegue y acceso al sistema	32
7.2.1	Acceso al portal para empresas (GitHub Pages)	32
7.2.2	Ejecución de la aplicación de usuario en local (Expo web)	33
7.2.3	Instalación de la aplicación en iPhone	33
7.3	Registro y acceso al sistema	34
7.4	Pantalla principal	34
7.5	Gestión de listas de la compra	35
7.6	Catálogo y comparativa de precios	39
7.7	Mapa de tiendas	41
7.8	Alertas de precio y notificaciones	43
7.9	Perfil y preferencias del usuario	43
7.10	Ruta de compra optimizada	46
7.11	Reconocimiento de tickets y listas (OCR)	47
7.12	Asistente conversacional	47
7.13	Lista de comprobación en pantalla de bloqueo (iOS)	48
7.14	La aplicación en el navegador de escritorio	49
7.15	Portal Business para PYMEs	50
7.16	Resolución de incidencias comunes	54
8	Pruebas y Resultados	55
8.1	Objetivo	55
8.2	Estrategia de testing	55
8.3	Entorno de ejecución	55
8.4	Suite de tests backend	56
8.4.1	Tests de integración	56
8.4.2	Cobertura	56
8.5	Tests E2E con Playwright	57
8.5.1	Configuración Playwright	57
8.6	UAT manual en dispositivo móvil	57
8.7	Validación de Requisitos No Funcionales	57
8.7.1	RNF-002: Disponibilidad del entorno de staging	57
8.7.2	RNF-004: Usabilidad y flujo máximo de 3 interacciones	58
8.7.3	RNF-005: Escalabilidad para 10.000 usuarios	58
8.8	Resumen de resultados	59
8.9	Conclusión	59
9	Conclusiones	60
9.1	Grado de cumplimiento	60
9.2	Aportaciones principales del trabajo	60
9.3	Limitaciones	61
9.4	Lecciones aprendidas	62
9.5	Trabajo futuro	62
9.6	Cierre	63

A Catálogo de requisitos	64
A.1 Requisitos de información	64
A.2 Requisitos funcionales	65
A.3 Requisitos no funcionales	66
A.4 Reglas de negocio	66
A.5 Trazabilidad requisitos–módulos–pruebas	67
B Relación de endpoints de la API	68
C Capturas de la versión de escritorio	70
Referencias	72

Índice de cuadros

3.1	Comparativa de capacidades clave (elaboración propia, marzo de 2026) . . .	6
4.1	Resumen por fases del proyecto	8
4.2	Coste estimado del desarrollo	10
5.1	Agrupación de requisitos funcionales	12
6.1	Índices principales de la base de datos	19
6.2	Endpoints de autenticación y usuarios	21
6.3	Endpoints del optimizador, OCR y asistente	21
6.4	Salvaguardas de rendimiento del optimizador	26
6.5	Paleta de colores del sistema de diseño (<code>src/theme/colors.ts</code>)	30
8.1	Suite de tests de integración backend	56
8.2	Suite de tests E2E con Playwright	57
8.3	Resultados UAT en dispositivo móvil	57
8.4	Resumen de resultados de la estrategia de testing	59
A.1	Catálogo de requisitos funcionales	65
A.2	Trazabilidad de dominios funcionales a módulos y suites de prueba	67
B.1	Endpoints de la API por módulo	68

Índice de figuras

5.1	Casos de uso del Consumidor	14
5.2	Casos de uso del Comercio y Administrador	15
6.1	Arquitectura por capas de BarGAIN	17
6.2	Diagrama entidad-relación de BarGAIN	20
6.3	Diagrama de clases del dominio	20
6.4	Diagrama de secuencia del proceso de optimización de ruta	25
6.5	Diagrama de secuencia del flujo OCR	28
7.1	Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)	35
7.2	Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio	36
7.3	Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)	37
7.4	Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada	38
7.5	Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)	39
7.6	Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador	40
7.7	Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)	41
7.8	Tiendas favoritas del usuario, con acceso directo a su perfil	42
7.9	Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)	43
7.10	Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del <i>scoring</i> multicriterio (derecha)	44
7.11	Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)	45
7.12	Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo	46
7.13	Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)	47
7.14	Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista	48
7.15	Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)	49
7.16	Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos	50

7.17 Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)	50
7.18 <i>Onboarding</i> del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta	51
7.19 Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio	52
7.20 Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)	52
7.21 Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad	53
7.22 Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)	53
7.23 Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto	54
C.1 Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)	70
C.2 Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)	70
C.3 Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)	71
C.4 Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)	71
C.5 Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)	71
C.6 Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio	71

Acrónimos

- ADR** *Architecture Decision Record*: registro de decisión arquitectónica.
- API** *Application Programming Interface*: interfaz de programación de aplicaciones.
- CI/CD** *Continuous Integration / Continuous Delivery*: integración y entrega continuas.
- CRUD** *Create, Read, Update, Delete*: operaciones básicas de persistencia.
- CSV** *Comma-Separated Values*: formato tabular de texto plano.
- DRF** *Django REST Framework*: framework para construir APIs REST sobre Django.
- E2E** *End-to-End*: pruebas de extremo a extremo del sistema integrado.
- EAN** *European Article Number*: código de barras normalizado de producto.
- GIN** *Generalized Inverted Index*: índice invertido generalizado de PostgreSQL.
- GiST** *Generalized Search Tree*: familia de índices de PostgreSQL usada por PostGIS.
- HMR** *Hot Module Replacement*: recarga en caliente de módulos en desarrollo.
- HU** Historia de Usuario.
- IA** Inteligencia Artificial.
- JSON** *JavaScript Object Notation*: formato de intercambio de datos.
- JWT** *JSON Web Token*: estándar de testigos de autenticación firmados.
- LLM** *Large Language Model*: modelo de lenguaje de gran tamaño.
- LSSI-CE** Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico.
- OCR** *Optical Character Recognition*: reconocimiento óptico de caracteres.
- ORM** *Object-Relational Mapping*: mapeo objeto-relacional.
- ORS** OpenRouteService: servicio de cálculo de rutas sobre OpenStreetMap.
- PYME** Pequeña y Mediana Empresa.
- REST** *Representational State Transfer*: estilo arquitectónico de APIs web.
- RF** Requisito Funcional.
- RGPD** Reglamento General de Protección de Datos (Reglamento (UE) 2016/679).
- RI** Requisito de Información.

RN Regla de Negocio.

RNF Requisito No Funcional.

SDK *Software Development Kit*: kit de desarrollo de software.

SPA *Single-Page Application*: aplicación web de página única.

SRID *Spatial Reference System Identifier*: identificador de sistema de referencia espacial.

TFG Trabajo Fin de Grado.

TSP *Travelling Salesman Problem*: problema del viajante de comercio.

TTL *Time To Live*: tiempo de vigencia de un dato.

UAT *User Acceptance Testing*: pruebas de aceptación de usuario.

UI *User Interface*: interfaz de usuario.

URL *Uniform Resource Locator*: localizador uniforme de recursos.

UX *User Experience*: experiencia de usuario.

VRP *Vehicle Routing Problem*: problema de enrutamiento de vehículos.

WCAG *Web Content Accessibility Guidelines*: pautas de accesibilidad para contenido web.

CAPÍTULO 1

Introducción y objetivos del proyecto

1.1— Contexto y motivación

El consumidor español afronta la compra cotidiana en un contexto de fuerte variabilidad de precios entre cadenas, ausencia de una fuente unificada y alto coste temporal para comparar ofertas manualmente. Aunque el ahorro potencial entre supermercados es significativo, en la práctica ese ahorro se pierde cuando no se considera el coste de desplazamiento ni el tiempo de ruta.

BarGAIN nace para resolver ese problema como una plataforma de compra inteligente que integra precio, distancia y tiempo en una única decisión. El sistema combina una app móvil para consumidores, un portal web para PYMEs, y un backend modular con capacidades geoespaciales, OCR y asistencia conversacional.

1.2— Problema abordado

Las soluciones actuales de comparación de supermercados suelen responder a la pregunta “qué tienda tiene el precio más bajo”, pero no a la pregunta completa “qué compra me conviene realmente según mi ubicación y mis preferencias”.

El problema de BarGAIN se formula como una optimización multicriterio en la que intervienen:

- coste total de la cesta,
- distancia adicional de desplazamiento,
- tiempo total estimado de compra y ruta.

1.3— Objetivo principal

Desarrollar una aplicación móvil y web que optimice la cesta de la compra del usuario, calculando la combinación de supermercados y comercios locales que ofrece la mejor relación precio–distancia–tiempo según preferencias configurables.

1.4— Objetivos específicos

1. Implementar una API REST con Django y DRF para usuarios, productos, tiendas, precios, listas, optimización, OCR, asistente y portal business.

2. Integrar una capa geoespacial con PostgreSQL + PostGIS para consultas de proximidad y preparación de rutas.
3. Desarrollar un optimizador multicriterio con OR-Tools y una función de scoring configurable por el usuario.
4. Incorporar OCR con Google Cloud Vision para digitalizar tickets y listas, con validación y corrección manual en frontend.
5. Integrar un asistente LLM (Gemini vía backend proxy) con guardrails de dominio y contexto de compra.
6. Construir un portal para PYMEs con gestión de precios y promociones.
7. Asegurar calidad con tests unitarios, de integración y E2E, además de UAT manual para flujos nativos.
8. Preparar despliegue reproducible en staging con Render, Docker Compose y CI.

1.5— Alcance del proyecto

El alcance del TFG cubre el ciclo completo de ingeniería del software:

- análisis y requisitos,
- diseño arquitectónico y de datos,
- implementación backend y frontend,
- pruebas funcionales y no funcionales,
- despliegue en entorno staging,
- documentación técnica y memoria final.

Quedan fuera del alcance de esta versión: despliegue productivo global, acuerdos formales con todas las cadenas para APIs oficiales, y la versión final de Live Activities nativas en iOS (ADR-010).

1.6— Tecnologías clave

El sistema se apoya en un stack full-stack moderno y coherente:

- **Backend:** Django 5, DRF, Celery, Redis.
- **Datos:** PostgreSQL 16 + PostGIS 3.4.
- **Frontend:** React Native con Expo + companion web en React.
- **IA:** Google Cloud Vision API (OCR) y Gemini vía backend proxy.
- **Optimización:** OR-Tools + cálculo de distancias (OpenRouteService + fallback haversine).
- **Ops:** Docker, GitHub Actions, Render, Sentry.

1.7— Estructura de la memoria

La memoria se organiza para reflejar el recorrido completo del proyecto: contexto y objetivos, estado del arte, comparativa, planificación, requisitos, diseño e implementación, manual de usuario, pruebas y resultados, y conclusiones. Cierran el documento los apéndices —con el catálogo completo de requisitos, la relación de endpoints de la API y las capturas de la versión de escritorio— y las referencias bibliográficas. Al inicio se incluye un glosario con los acrónimos empleados.

Análisis de antecedentes y aportación realizada

2.1— Contexto del sector

El mercado de distribución alimentaria en España presenta una combinación de gran competencia entre cadenas, alta sensibilidad al precio y fuerte fragmentación de oferta. Durante los últimos años, la evolución al alza del índice de precios de la alimentación registrada por el IPC (Instituto Nacional de Estadística, 2026) ha incrementado la necesidad de herramientas que permitan comprar mejor sin aumentar el tiempo invertido.

Aunque existen comparadores de precios consolidados, la mayor parte se centra en mostrar información de coste por tienda, sin integrar de forma nativa el impacto logístico de desplazarse a varios establecimientos.

2.2— Estado del arte en comparación de precios

La evolución observada puede resumirse en tres generaciones:

1. **Directorios de precios:** listados estáticos o semiactualizados, con poca cobertura y baja frecuencia de refresco.
2. **Comparadores multisupermercado:** soluciones como Soysuper (Soysuper S.L., 2026), Findit u OCU Market (Organización de Consumidores y Usuarios (OCU), 2026), con mejor catálogo y funcionalidades de lista.
3. **Personalización con IA:** tendencia emergente en la que se introducen recomendaciones contextuales y asistencia conversacional.

En España, el salto completo hacia la tercera generación todavía es limitado, especialmente en el segmento de cesta alimentaria diaria.

2.3— Tecnologías relevantes para BarGAIN

2.3.1. Geoespacial y rutas

PostGIS (PostGIS Project Steering Committee, 2026; Obe y Hsu, 2021) permite resolver consultas de proximidad con precisión y rendimiento, mientras que OR-Tools (Google OR-Tools Team, 2026) facilita modelar el problema de ordenación de paradas como una variante del TSP/VRP (Applegate, Bixby, Chvatal, y Cook, 2006) en instancias pequeñas (máximo 3–4 paradas en nuestro caso). Para el cálculo de distancias reales por carretera existen servicios de enrutamiento sobre datos de OpenStreetMap (Luxen y Vetter, 2011),

de los que el proyecto adopta OpenRouteService (Heidelberg Institute for Geoinformation Technology (HeiGIT), 2026).

2.3.2. Ingesta de datos de precio

La combinación de Scrapy (Scrapy Developers, 2026) y Playwright (Microsoft Playwright Team, 2026) es adecuada para catálogos dinámicos de supermercados. Sin embargo, por robustez de producto, BarGAIN no depende de una única fuente y combina:

- scraping programado,
- aportaciones de crowdsourcing,
- actualización manual de comercios verificados.

2.3.3. OCR y asistente

Para OCR se adopta Google Cloud Vision API (Google Cloud, 2026) por mejor rendimiento práctico en tickets y fotos reales. Para el asistente se integra Gemini (Google, 2026) mediante backend proxy, con guardrails para limitar respuestas al dominio de la compra. La viabilidad de este tipo de asistentes se apoya en la capacidad de los grandes modelos de lenguaje para resolver tareas con pocos ejemplos (Brown y cols., 2020) y en técnicas de generación aumentada con datos externos (Lewis y cols., 2020), que en BarGAIN se concreta en inyectar el contexto real de precios y tiendas del usuario en cada consulta.

2.4– Aportación diferencial de BarGAIN

La aportación del proyecto se concreta en cuatro ejes:

- **Optimización multicriterio real** (precio + distancia + tiempo) en lugar de comparación aislada de precios.
- **Inclusión de comercio local** mediante portal business y actualización de precios/promociones.
- **Digitalización de lista** por OCR con validación de usuario.
- **Interacción conversacional** para consultas complejas de compra.

Estas capacidades, integradas en una sola plataforma, posicionan BarGAIN en un espacio de mercado poco cubierto por herramientas actuales.

CAPÍTULO 3

Comparación con otras alternativas

3.1— Competidores considerados

Para el análisis competitivo se han considerado cinco soluciones de referencia en el mercado español de comparación de compra:

- Soysuper (Soysuper S.L., 2026)
- Findit
- OCU Market (Organizacion de Consumidores y Usuarios (OCU), 2026)
- PreciRadar
- RadarPrice

Adicionalmente, se han valorado como competencia indirecta las apps nativas de cadenas y como sustituto no tecnológico la comparación manual con folletos. La revisión de capacidades se realizó sobre las versiones públicas de cada herramienta durante la fase de análisis del proyecto (marzo de 2026), por lo que la matriz refleja el estado de las soluciones en esa fecha.

3.2— Matriz funcional resumida

El Cuadro 3.1 resume la comparativa de capacidades clave; el análisis extendido, característica a característica, se mantiene en la documentación del repositorio (`docs/memoria/04-comparativa.m`).

Cuadro 3.1: Comparativa de capacidades clave (elaboración propia, marzo de 2026)

Capacidad	BarGAIN	Soysuper	OCU Market	Resto
Comparación multi-tienda de cesta	Completa	Completa	Parcial	Parcial
Optimización de ruta (precio + distancia + tiempo)	Completa	No	No	No
Integración de PYMEs / comercio local	Completa	No	No	No
OCR de lista / ticket	Completa	No	No	No
Asistente conversacional IA	Completa	No	No	No
Histórico avanzado de precios	Parcial	Limitado	Limitado	Variable

3.3– Brechas estratégicas detectadas

El análisis identifica cuatro brechas de mercado que justifican la propuesta:

1. **Gap precio-ruta:** los comparadores muestran precios, pero no optimizan desplazamiento y tiempo.
2. **Comercio local invisible:** no existe cobertura estable de PYMEs en herramientas generalistas.
3. **Fricción de entrada:** la lista de compra sigue siendo manual en la mayoría de soluciones.
4. **Interacción limitada:** predominan filtros y búsquedas rígidas frente a consultas en lenguaje natural.

3.4– Posicionamiento de BarGAIN

BarGAIN se posiciona como un *optimizador inteligente de compra* y no solo como comparador. Su propuesta de valor combina ahorro económico, eficiencia logística y experiencia de usuario asistida por IA.

3.5– Riesgos competitivos

Los riesgos principales identificados son:

- reacción de competidores consolidados con mayor base de usuarios,
- dependencia de fuentes de datos de terceros para scraping,
- necesidad de mantener alta calidad de datos para sostener confianza.

Como mitigación, se adopta una estrategia de fuentes híbridas de datos, modularidad arquitectónica y pipeline de validación continua.

Análisis temporal y costes de desarrollo

4.1— Planificación temporal

La planificación de BarGAIN se definió con una dedicación planificada de 300 horas, distribuidas en fases funcionales y técnicas, que se amplió hasta 325 horas reales al incorporar el cierre documental y la preparación de la defensa. El proyecto se ejecutó de forma iterativa, manteniendo trazabilidad en `TASKS.md` y en el árbol `.planning/`.

Cuadro 4.1: Resumen por fases del proyecto

Fase	Semanas	Horas	Estado
F1 · Análisis y diseño	S1–S3	45	Completada
F2 · Infraestructura base	S3–S5	30	Completada
F3 · Desarrollo core backend	S5–S12	95	Completada
F4 · Desarrollo frontend	S8–S16	70	Completada
F5 · IA, optimizador y scraping	S10–S17	35	Completada
F6 · Portal business y app móvil	S15–S18	25	Completada
F7 · Pruebas, deploy y cierre	S18–S20	25	Completada
Total		325	Cerrado

4.2— Metodología de trabajo y trazabilidad del proceso

Se aplicó una metodología incremental con ciclos cortos por fase —definición de alcance y criterios de aceptación, implementación por módulos, validación automatizada (lint y tests), verificación funcional y documental, y cierre de fase con artefactos de evidencia— que permitió absorber cambios sin comprometer la estabilidad del sistema. Sobre esa base incremental, el proyecto incorpora una característica metodológica singular que merece descripción propia: todo el ciclo de vida se ejecutó mediante un flujo de desarrollo asistido por agentes de inteligencia artificial, gobernado por el método *GSD* (*Get Stuff Done*) y versionado íntegramente en el propio repositorio. El resultado es que no solo el producto, sino también el *proceso* de ingeniería que lo produjo, es inspeccionable, auditable y reproducible por terceros.

4.2.1. Planificación viva: el árbol `.planning/`

El método GSD sustituye los documentos de planificación estáticos por un árbol de ficheros Markdown versionados que evolucionan con el proyecto. En `.planning/` conviv-

en la definición del producto (`PROJECT.md`), los requisitos operativos (`REQUIREMENTS.md`), la hoja de ruta por fases (`ROADMAP.md`) y una instantánea del estado actual (`STATE.md`, `STATUS.md`) que cualquier agente —o persona— lee antes de actuar. Cada fase se descompone en `.planning/phases/` en pares `PLAN/SUMMARY`: el plan fija el alcance y los criterios de aceptación *antes* de implementar, y el resumen registra lo realmente ejecutado, las desviaciones y las evidencias de verificación al cerrar. El subdirectorio `.planning/debug/` funciona como base de conocimiento de incidencias: cada problema no trivial (un 404 de CORS en GitHub Pages, un archivo iOS que no se generaba, la conectividad móvil contra Render) queda documentado con su diagnóstico y resolución, y se mueve a `resolved/` al cerrarse.

4.2.2. Trazabilidad de tareas y evidencias

La fuente de verdad del seguimiento es `TASKS.md`, un tablero versionado que asigna a cada tarea un identificador estable (`F3-01`, `F7-08...`), su estado, horas estimadas y reales, y el entregable que produce. La convención de *commits* exige referenciar ese identificador (`feat(optimizer): ... (F5-06)`), de modo que cualquier línea de código es trazable hacia atrás —hasta la tarea, la fase y el requisito que la motivaron— y hacia delante —hasta el test y la evidencia que la verifican—. El tablero se sincroniza además con GitHub Issues y con un *backlog* en Notion, manteniendo una única numeración. Esta disciplina, habitual en equipos industriales pero infrecuente en un TFG individual, fue la que permitió absorber 325 horas de trabajo en 20 semanas sin pérdida de control del alcance.

4.2.3. Agentes de IA con contrato de contexto explícito

Los agentes (Claude, GitHub Copilot y Codex) no operan sobre instrucciones improvisadas: el repositorio define un *contrato de contexto* común. El fichero `CLAUDE.md` reúne la arquitectura, las convenciones de código, las reglas de negocio y el protocolo obligatorio que todo agente debe seguir antes, durante y después de cada tarea; los directorios `.github/agents/`, `.github/instructions/` y `.agent/workflows/` versionan agentes especializados (orquestador, ejecutor y revisor de fase), instrucciones por tecnología y flujos de trabajo reutilizables; y `docs/ai-prompts/` contiene las plantillas de inicio, cierre, revisión y sincronización de tareas —el equivalente a los *runbooks* de operaciones—, duplicadas como *prompts* de Copilot para mantener paridad entre herramientas. Las *skills* empleadas durante el desarrollo se versionan igualmente junto al código, de manera que el entorno de trabajo de los agentes forma parte del entregable.

4.2.4. Registro de errores y reglas derivadas

El componente más distintivo del flujo es `docs/ai-mistakes-log.md`: cada error cometido por un agente se registra con un formato fijo —contexto, error, causa raíz, solución aplicada y prevención— y, cuando procede, se destila en una *regla derivada* numerada que todos los agentes releen obligatoriamente al iniciar una tarea. Así, por ejemplo, la regla que prohíbe entregar diagramas de interfaz en ASCII o la que obliga a validar la existencia de rutas de sistema antes de aplicarlas en *settings* compartidos nacieron de errores reales y han evitado su repetición. El registro convierte los fallos de los agentes en conocimiento acumulativo del proyecto: el proceso *aprende* de manera verificable, y el historial de ese aprendizaje queda en el control de versiones.

4.2.5. Valoración del enfoque

Conviene delimitar el reparto de responsabilidades: los agentes ejecutan, pero la definición de fases, los criterios de aceptación, la revisión de cada entrega y la decisión final son

del autor. La combinación resultó eficaz por dos razones. Primera, la productividad: tareas mecánicas (generación de tests, capturas, scripts de despliegue) se delegaron sin pérdida de calidad gracias al contrato de contexto. Segunda, la fiabilidad del proceso: la planificación viva y el registro de errores con reglas derivadas impusieron al trabajo con IA la misma disciplina de evidencias que se exigiría a un equipo humano. El enfoque es, además, reproducible: cualquier evaluador puede clonar el repositorio y reconstruir no solo el sistema, sino la historia completa de decisiones que lo produjo.

4.3— Estimación de costes

La estimación económica se realizó mediante coste-hora por rol junior, tomando como referencia el informe de perfiles profesionales TIC de la Junta de Andalucía (Junta de Andalucía, 2018) (columna de media acotada para perfiles con menos de seis años de experiencia). Conviene señalar que se trata de la referencia pública más completa disponible para tarifas por rol en Andalucía pero data de 2018, por lo que las tarifas aplicadas deben entenderse como una cota conservadora: los salarios TIC de mercado en 2026 son superiores, de modo que el coste real de un desarrollo equivalente sería previsiblemente mayor que el estimado.

Cuadro 4.2: Coste estimado del desarrollo

Bloque	Horas	Coste (EUR)
Análisis y diseño	45	1.490,40
Infraestructura	30	861,60
Backend + frontend	165	4.738,80
IA, optimizador y scraping	35	1.005,20
Portal business y app móvil	25	718,00
Pruebas y despliegue	25	594,25
Total	325	9.408,25

Adicionalmente, el coste operativo en staging se mantuvo bajo durante el TFG gracias al uso de planes gratuitos o de bajo consumo (Render, Sentry, GitHub Actions).

4.4— Riesgos y mitigaciones

Los riesgos operativos principales fueron:

- **Cambios en fuentes de scraping:** mitigado con spiders modulares y fallback de crowdsourcing.
- **Dependencia de APIs externas:** mitigado con caché y degradación controlada.
- **Carga de trabajo en fases finales:** mitigado con cierre incremental de entregables y automatización de validación.

4.5— Resultado de planificación

La planificación permitió completar el alcance funcional priorizado del TFG, incluyendo arquitectura, implementación, pruebas y despliegue de staging, manteniendo coherencia entre roadmap, requisitos y evidencias de verificación.

Análisis de requisitos

5.1— Actores del sistema

El sistema define cuatro actores principales:

- **Consumidor:** crea listas, compara precios, optimiza rutas, usa OCR y consulta al asistente.
- **Comercio/PYME:** gestiona precios y promociones desde el portal business.
- **Administrador:** verifica perfiles, modera datos y supervisa la operación.
- **Sistema automatizado:** tareas de scraping y mantenimiento en segundo plano.

5.2— Requisitos de información

El modelo de información se articula en doce bloques (RI-001 a RI-012), que cubren:

- usuarios y preferencias de optimización,
- catálogo (productos, categorías, marcas),
- tiendas geolocalizadas y cadenas,
- precios actuales e históricos,
- listas de compra y resultados de optimización,
- perfiles business y promociones,
- sesiones OCR y conversaciones del asistente.

5.3— Requisitos funcionales

Se definieron 35 requisitos funcionales (RF-001 a RF-035) agrupados por dominios, tal y como resume el Cuadro 5.1. El catálogo completo, con el enunciado de cada requisito, se recoge en el Apéndice A.

Cuadro 5.1: Agrupación de requisitos funcionales

Dominio	IDs	Capacidad principal
Autenticación y perfil	RF-001..RF-005	Registro, login JWT, preferencias
Catálogo de productos	RF-006..RF-010	Búsqueda, detalle, crowdsourcing
Tiendas y mapa	RF-011..RF-014	Proximidad, detalle, favoritos
Precios	RF-015..RF-019	Comparación, histórico, alertas
Listas de compra	RF-020..RF-023	CRUD, compartir, plantillas
Optimizador	RF-024..RF-027	Ruta multicriterio y recálculo
OCR	RF-028..RF-029	Captura y matching fuzzy
Asistente LLM	RF-030..RF-031	Consultas naturales y sugerencias
Portal business	RF-032..RF-034	Onboarding, precios, promociones
Notificaciones	RF-035	Push / email por eventos

5.4— Criterios de aceptación del producto

Los criterios de aceptación se validan a nivel funcional y de experiencia:

- flujo de alta y acceso operativo en menos de 3 minutos,
- comparación de cesta completa en entorno de tiendas cercanas,
- optimización con ruta recomendada, desglose por parada y resolución de ambigüedades semánticas con recálculo,
- OCR con confirmación/edición previa a la inserción en lista,
- portal PYME con gestión de precios y promociones,
- notificaciones de eventos críticos (alertas, promociones, cambios compartidos).

5.5— Requisitos no funcionales

Se definieron ocho requisitos no funcionales (RNF-001 a RNF-008), cuyo enunciado completo se recoge en el Apéndice A:

- **RNF-001** rendimiento API (p95 inferior a 500 ms en CRUD y a 5 s en optimización),
- **RNF-002** disponibilidad del entorno de staging,
- **RNF-003** seguridad (HTTPS, JWT con rotación, rate limiting, hashing de contraseñas),
- **RNF-004** usabilidad (flujo principal en un máximo de 3 interacciones y criterios de accesibilidad WCAG 2.1 (World Wide Web Consortium (W3C), 2018)),
- **RNF-005** escalabilidad arquitectónica,
- **RNF-006** mantenibilidad (cobertura de tests ≥ 80 % y convenciones verificadas en CI),
- **RNF-007** compatibilidad multiplataforma,
- **RNF-008** protección de datos conforme al RGPD (Parlamento Europeo y Consejo de la Union Europea, 2016).

5.6— Reglas de negocio

Las reglas RN-001..RN-010 gobiernan límites y políticas operativas:

- radio máximo de búsqueda y número máximo de paradas,
- caducidad y prioridad de fuentes de precio,
- restricciones del asistente al dominio de compra,
- verificación previa de perfiles business,
- políticas de scraping responsable y frecuencia de ejecución.

5.7— Diagramas de casos de uso

Los diagramas de casos de uso recogen las interacciones principales de cada actor con el sistema.

5.8— Trazabilidad

La trazabilidad entre requisitos, implementación y pruebas se documenta en tres niveles:

- el Apéndice A de esta memoria (catálogo completo de requisitos y tabla de trazabilidad dominio–módulo–suite de pruebas),
- `TASKS.md` en el repositorio (estado por fase/tarea, con identificadores referenciados desde los commits),
- `tests/unit`, `tests/integration` y `frontend/web/e2e` (verificación ejecutable).

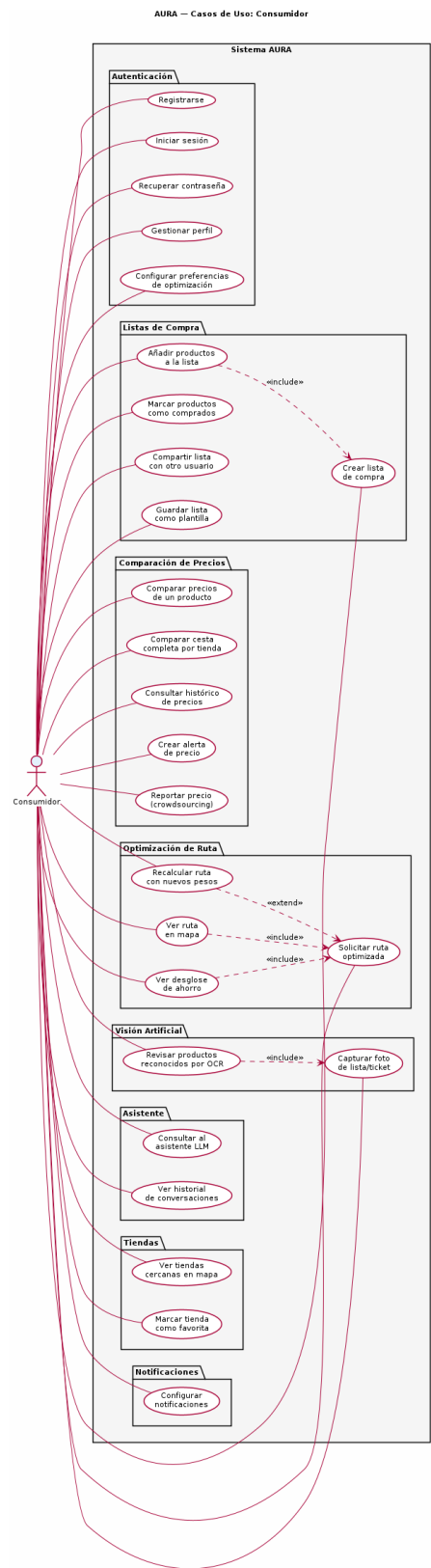


Figura 5.1: Casos de uso del Consumidor

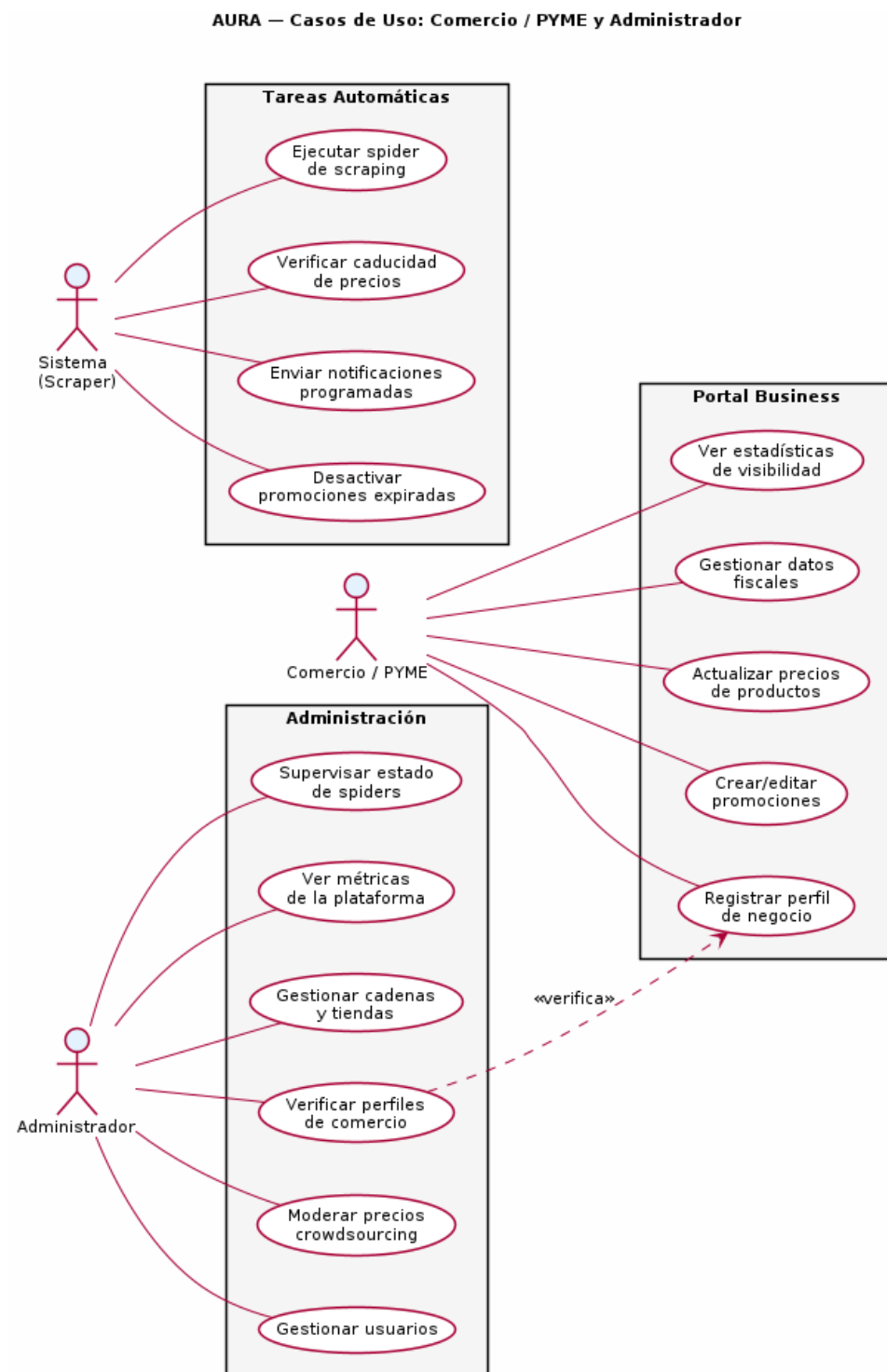


Figura 5.2: Casos de uso del Comercio y Administrador

Diseño e Implementación

6.1— Arquitectura del Sistema

6.1.1. Visión general

BarGAIN sigue una arquitectura de **tres capas** clásica (presentación, lógica de negocio y datos), desplegada mediante contenedores Docker y organizada internamente como una **arquitectura en capas con capa de servicios explícita** por aplicación Django (Django Software Foundation, 2026; Encode OSS Ltd., 2026): las vistas se limitan a la gestión HTTP y la lógica de dominio reside en módulos de servicio independientes. Esta separación desacopla la lógica de negocio de los detalles de infraestructura — bases de datos, APIs externas, brokers de mensajería — y facilita el testing independiente de cada componente.

La arquitectura global se divide en cinco bloques principales:

1. **Aplicación cliente:** React Native (Expo) para iOS, con una aplicación web React para el Portal Business de PYMEs.
2. **API Backend:** Django REST Framework como núcleo de la lógica de negocio, expuesto bajo HTTPS mediante Gunicorn + Nginx.
3. **Capa de datos:** PostgreSQL 16 con extensión PostGIS 3.4 para consultas geoespaciales.
4. **Capa de tareas asíncronas:** Celery con Redis como broker, para scraping periódico, envío de notificaciones, procesamiento OCR y cálculos del optimizador.
5. **Integraciones y servicios auxiliares:** Google Gemini API (asistente LLM), OpenRouteService (cálculo de distancias reales), Google Cloud Vision API (OCR) y Sentry (monitorización de errores).

6.1.2. Patrón de organización interna del backend

Cada aplicación Django del backend sigue una estructura modular consistente:

```
apps/<modulo>/
|-- models.py           # Entidades de dominio (ORM)
|-- serializers.py       # Transformacion de datos (entrada/salida)
|-- views.py            # ViewSets y vistas (controladores)
|-- urls.py             # Rutas del modulo
|-- services.py         # Logica de negocio
```

```

|-- tasks.py           # Tareas Celery asincronas
|-- permissions.py     # Logica de autorizacion
'-- tests/

```

La separación entre `views.py` y `services.py` es deliberada: los ViewSets se limitan a la gestión HTTP (autenticación, serialización, códigos de respuesta), mientras que la lógica de negocio reside exclusivamente en los servicios, lo que permite invocarla desde tareas Celery o tests sin simular peticiones HTTP. En los módulos de mayor complejidad (`optimizer`, por ejemplo) `services.py` se sustituye por un paquete `services/` con varios módulos especializados (`matching`, `semantic`, `distance`, `solver`).

La Figura 6.1 resume la distribución por capas de la solución.

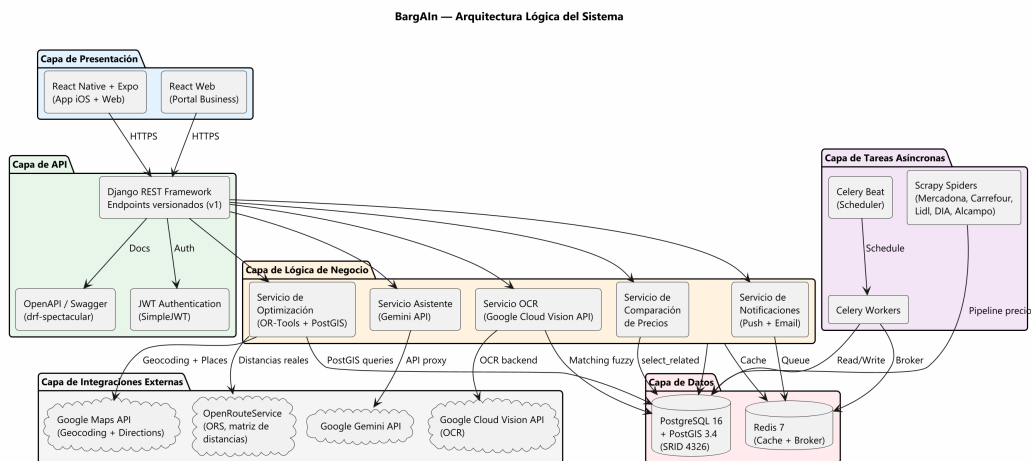


Figura 6.1: Arquitectura por capas de BarGAIN

6.1.3. Comunicación entre componentes

- **Frontend → Backend:** API REST con autenticación JWT. Axios gestiona la renovación automática del token de acceso ante un `401 Unauthorized`.
- **Backend → Celery:** Encolar tareas mediante `.delay()` o `.apply_async()` sobre el broker Redis.
- **Backend → PostgreSQL:** Django ORM con consultas geoespaciales mediante `django.contrib.gis`.
- **Backend → APIs externas:** Clientes HTTP aislados en la capa de servicios de cada módulo (p.ej. `apps/optimizer/services/distance.py`) para Google Places, Gemini API y OpenRouteService.

6.2— Modelo de Datos

6.2.1. Módulo de usuarios

El modelo `User` extiende `AbstractUser` de Django, conservando el nombre de usuario como identificador de inicio de sesión; el correo electrónico se valida como único en el registro y se emplea para las comunicaciones y la recuperación de contraseña. Se definen tres roles: `consumer`, `business` y `admin`. El campo `default_location` utiliza `PointField(SRID=4326)`.

Las preferencias de optimización se almacenan en el perfil como enteros porcentuales (`weight_price`, `weight_distance`, `weight_time`, con valores por defecto 34/33/33):

```
weight_price = models.PositiveSmallIntegerField(
    default=34,
    verbose_name="Peso precio (%)",
    help_text="Importancia del precio en la optimizacion (0-100). "
              "Junto a weight_distance y weight_time deben sumar ~100.",
)
```

La interfaz ajusta los tres deslizadores de forma conjunta y, en cada petición de optimización, el serializer del optimizador renormaliza los pesos recibidos dividiéndolos por su suma, de modo que el algoritmo siempre trabaja con pesos relativos que suman 1 con independencia del formato de entrada.

6.2.2. Módulo de productos

La búsqueda de productos por nombre utiliza **matching fuzzy** mediante la extensión PostgreSQL `pg_trgm` (trigramas). El campo `normalized_name` almacena el nombre normalizado (minúsculas, sin tildes, sin caracteres especiales) con índices GIN de trigramas:

```
queryset.annotate(
    similarity=TrigramSimilarity("normalized_name", value)
).filter(similarity__gte=0.3).order_by("-similarity")
```

6.2.3. Módulo de tiendas

El campo `location` de `Store` es un `PointField` con índice espacial GiST (PostGIS Project Steering Committee, 2026). La búsqueda por radio se implementa con el *lookup distance_lte* de GeoDjango y la anotación `Distance` para ordenar por cercanía:

```
user_location = Point(lng, lat, srid=4326)
stores = (
    Store.objects
    .filter(
        location__distance_lte=(user_location, D(km=radius_km)),
        is_active=True,
    )
    .annotate(distance=Distance("location", user_location))
    .order_by("distance")
)
```

Para los volúmenes de tiendas manejados en el TFG (decenas por radio) esta consulta es suficiente. Como optimización identificada para escenarios con cientos de miles de tiendas, el *lookup dwithin* (que se traduce en `ST_DWithin` y explota de forma más directa el índice GiST (Obe y Hsu, 2021)) sustituiría a `distance_lte` en el filtro, manteniendo `Distance` solo para la ordenación.

6.2.4. Módulo de precios

El histórico de precios se materializa automáticamente: cada precio es inmutable una vez creado. La actualización de un precio supone la inserción de un nuevo registro. Una tarea Celery periódica (`expire_stale_prices`, ejecutada cada hora) marca como caducados —bandera `is_stale`— los precios de scraping con más de 48 horas de antigüedad y los de crowdsourcing con más de 24 horas, conforme a la regla de negocio RN-003.

6.2.5. Módulo del optimizador

El módulo del optimizador incluye tres entidades relacionales para almacenar el resultado de la optimización de forma trazable:

- **OptimizationResult:** resultado global con pesos, modo y métricas totales. Incluye `route_data` JSONB para compatibilidad de lectura rápida.
- **OptimizationRouteStop:** cada parada de la ruta con tienda, orden, distancia, tiempo y subtotal.
- **OptimizationRouteStopItem:** ítem concreto por parada, con producto resuelto, precio elegido, texto original y puntuación de similitud.

6.2.6. Índices clave

El Cuadro 6.1 recoge los índices definidos explícitamente en los modelos, además de los índices automáticos de claves primarias y foráneas.

Cuadro 6.1: Índices principales de la base de datos

Tabla	Columna(s)	Tipo	Justificación
store	location	GiST	Búsquedas geoespaciales por radio
product	normalized_name	GIN (trigram)	Búsqueda fuzzy por nombre
price	(product, store, is_stale)	B-Tree	Precio vigente por par producto-tienda

6.2.7. Modelo entidad-relación

La Figura 6.2 muestra el diagrama entidad-relación completo de la base de datos.

6.2.8. Diagrama de clases

La Figura 6.3 detalla las clases principales del dominio y sus relaciones.

6.3— Diseño de la API REST

6.3.1. Estructura general

La API sigue las convenciones REST y está versionada bajo el prefijo `/api/v1/`. Todos los endpoints requieren autenticación JWT salvo los de registro, obtención de token, salud y recuperación de contraseña. Las vistas devuelven el *envelope* de éxito `{success, data}` (helpers de `apps/core/responses.py`) y los errores se normalizan globalmente mediante un *exception handler* propio (`apps/core/exceptions.py`):

```
// Respuesta exitosa
{
  "success": true,
  "data": { ... }
}

// Respuesta de error
{
  "success": false,
```

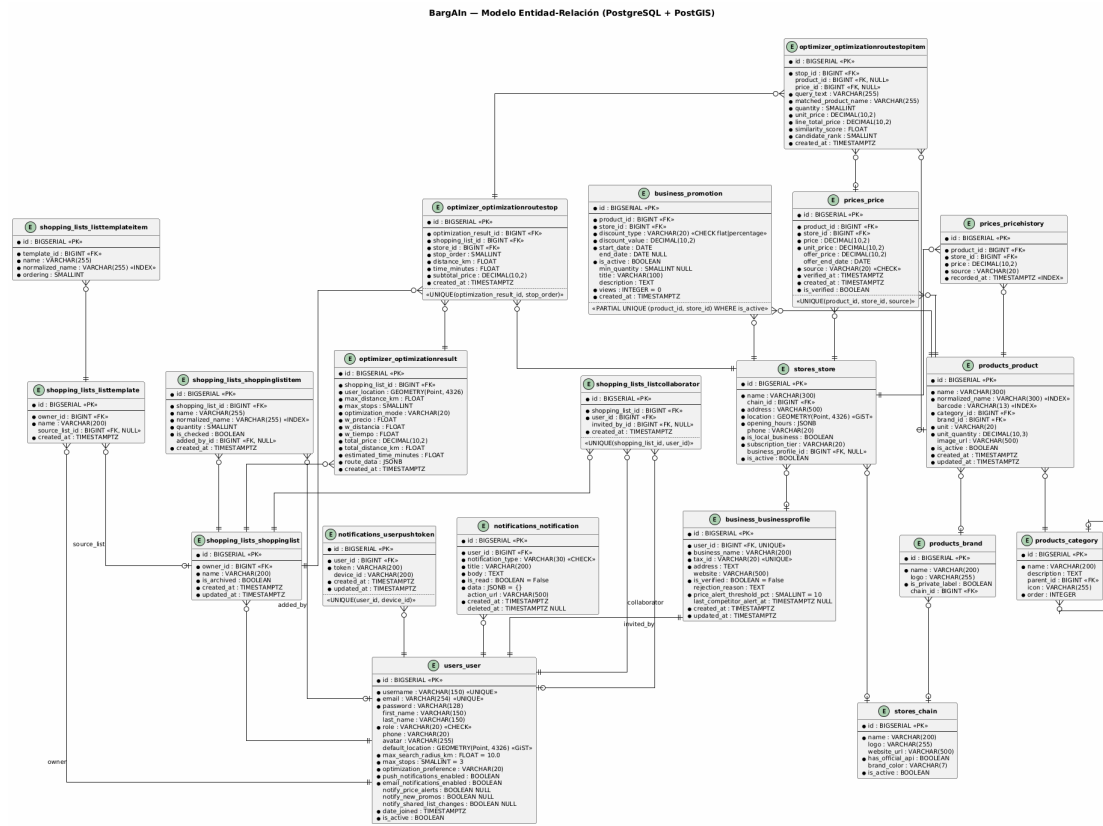


Figura 6.2: Diagrama entidad-relación de BarGAIN

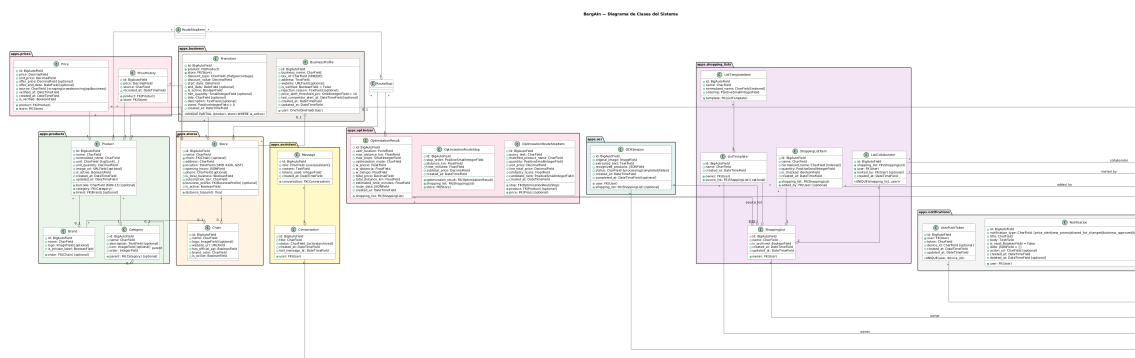


Figura 6.3: Diagrama de clases del dominio

```

"error": {
  "code": "STORE_NOT_FOUND",
  "message": "No se encontraron tiendas en el radio especificado",
  "details": {}
}
}

```

Los listados emplean la paginación estándar de DRF (`PageNumberPagination`), cuyo bloque `count/next/previous/results` viaja dentro del campo `data`.

6.3.2. Endpoints principales

Los Cuadros 6.2 y 6.3 recogen los endpoints de los dominios de autenticación y de las capacidades diferenciales; la relación completa por módulo se incluye en el Apéndice B.

Cuadro 6.2: Endpoints de autenticación y usuarios

Método	Endpoint	Descripción
POST	<code>/api/v1/auth/register/</code>	Registro de nuevo usuario
POST	<code>/api/v1/auth/token/</code>	Login: access + refresh token
POST	<code>/api/v1/auth/token/refresh/</code>	Renovar access token
POST	<code>/api/v1/auth/password-reset/</code>	Solicitar restablecimiento
POST	<code>/api/v1/auth/password-reset/confirm/</code>	Confirmar restablecimiento
GET/PATCH	<code>/api/v1/auth/profile/</code>	Perfil y preferencias del usuario

Cuadro 6.3: Endpoints del optimizador, OCR y asistente

Método	Endpoint	Descripción
POST	<code>/api/v1/optimize/</code>	Calcular o recalcular la ruta óptima
GET	<code>/api/v1/optimize/?shopping_list_id={id}</code>	Cargar última ruta persistida
POST	<code>/api/v1/optimize/choices/</code>	Resolver ambigüedad semántica y recalcular
POST	<code>/api/v1/ocr/scan/</code>	Procesar imagen y devolver ítems
POST	<code>/api/v1/assistant/chat/</code>	Enviar mensaje al asistente

6.3.3. Autenticación y autorización

Se implementa un sistema de permisos a tres niveles:

- **IsAuthenticated:** Aplicado por defecto a todos los endpoints privados.
- **IsBusinessOwner:** Permite operar solo sobre el perfil de negocio propio.
- **IsAdminUser:** Para endpoints de administración del sistema.
- **IsShoppingListOwnerOrShared:** Permite acceso al propietario y colaboradores invitados.

Los tokens JWT se generan con `django-rest-framework-simplejwt` (Jazzband, 2026), con rotación automática del token de refresco y lista negra del token rotado (`ROTATE_REFRESH_TOKENS` y `BLACKLIST_AFTER_ROTATION`). Los tiempos de vida son configurables por variables de entorno: la configuración de referencia del proyecto (`.env.example`) fija 60 minutos para el token de acceso y 7 días para el de refresco, mientras que los valores por defecto del código, más restrictivos, son de 5 minutos y 30 días respectivamente.

6.4— Implementación del Backend

6.4.1. Módulo de notificaciones

El módulo gestiona el buzón de notificaciones del usuario (inbox) y el envío de notificaciones push mediante **Expo Push Notifications**. Las entregas se procesan asincrónicamente con tareas Celery.

- **Notification:** Registro persistente con `notification_type` (`price_alert`, `new_promo`, `shared_list_changed`, `business_approved`, `business_rejected`), campos `title/body`, bandera `is_read`, data JSONB para payload variable, `action_url` para deep links y `deleted_at` para soft-delete.
- **UserPushToken:** Token Expo registrado por dispositivo. La clave única (`user`, `device_id`) evita duplicados al reinstalar la app.

6.4.2. Módulo de portal business

- **BusinessProfile:** Perfil verificable con `tax_id` (CIF/NIF único), estado de verificación y umbral de alerta de competidor (`price_alert_threshold_pct`).
- **Promotion:** Promoción temporal de un producto en una tienda. Restricción parcial única (`product`, `store`) WHERE `is_active` garantiza que no existan dos promociones activas para el mismo par.

La tarea Celery `check_competitor_prices` se ejecuta una vez al día (a las 08:00, según el *beat schedule*) y alerta al propietario cuando detecta diferencias superiores al umbral configurado.

6.5— El Algoritmo de Optimización de Rutas

6.5.1. Formulación del problema

El algoritmo de optimización es la aportación técnica central del TFG. El problema combina dos subproblemas acoplados: (1) la *asignación* de cada ítem textual de la lista a un producto concreto del catálogo y a la tienda donde comprarlo, y (2) la *ordenación* de las tiendas seleccionadas como una variante del problema del viajante (TSP) (Applegate y cols., 2006) con un término de premio por ahorro en cada parada.

Entrada:

- Lista de la compra $L = \{i_1, i_2, \dots, i_m\}$ (m ítems textuales)
- Ubicación del usuario $u = (lat, lng)$
- Radio máximo r (km), número máximo de paradas k (1–4)
- Pesos de preferencia w_{precio} , $w_{distancia}$, w_{tiempo} (renormalizados en el serializer para que sumen 1)

Salida: una asignación ítem $\rightarrow$ (producto, precio, tienda), la ruta recomendada con las paradas ordenadas y su desglose (precio total, distancia, tiempo y productos por parada), junto con las ambigüedades semánticas detectadas, que el usuario puede resolver para recalcular.

6.5.2. Fase 1 — Resolución semántico-difusa de ítems

Cada ítem textual se resuelve en `services/matching.py` mediante una cascada de filtros:

1. **Preselección por trigramas:** los 20 productos más similares por `TrigramSimilarity` sobre `normalized_name` (índice GIN (PostgreSQL Global Development Group, 2026)).
2. **Intención semántica (Gemini):** el módulo `services/semantic.py` consulta un modelo Gemini (Google, 2026) con el texto del ítem y los candidatos, y obtiene productos *preferidos* y *alternativos*, junto con la marca `needs_user_confirmation` cuando el ítem es genérico (“manzanas” frente a “manzanas Fuji”). Si el modelo no está disponible, se degrada a una heurística local basada en calificadores transformadores (“rallado”, “molido”...). Las preferencias que el usuario confirmó en optimizaciones anteriores (`ShoppingListSemanticPreference`) tienen prioridad sobre la inferencia.
3. **Precios vigentes:** una única consulta con `select_related` recupera el último precio no caducado (`is_stale=False`) de cada par producto–tienda dentro del radio.
4. **Puntuación difusa:** cada candidato (ítem, precio) se puntúa con `thefuzz` (SeatGeek, 2026) combinando `token_set_ratio` y `partial_ratio` sobre variantes del nombre (85 % producto, 15 % contexto de tienda), con una penalización para calificadores no solicitados y un bonus semántico (+0,15 preferidos, +0,05 alternativos). Se conservan los 12 mejores candidatos por ítem.
5. **Selección:** entre los candidatos cuya similitud queda a menos de 0,08 del máximo, se elige el de menor precio efectivo (precio de oferta si existe).

6.5.3. Fase 2 — Consolidación de paradas

Si la asignación inicial usa más tiendas que el máximo de paradas k , una reducción greedy recoloca ítems de tiendas con un solo producto hacia tiendas ya seleccionadas, eligiendo en cada paso el cambio de menor sobrecoste. Adicionalmente, si varias tiendas de una misma cadena participan en la ruta, se consolidan en una única tienda de esa cadena siempre que exista una alternativa sin sobrecoste.

6.5.4. Fase 3 — Matriz de distancias y tiempos

Las distancias y tiempos reales de conducción entre el usuario y las tiendas seleccionadas se obtienen de **OpenRouteService (ORS)** (Heidelberg Institute for Geoinformation Technology (HeiGIT), 2026) en una única petición de matriz; si la clave API no está configurada o la llamada falla, se aplica un *fallback* haversine con una velocidad urbana media de 40 km/h:

```
POST https://api.openrouteservice.org/v2/matrix/driving-car
Authorization: {ORS_API_KEY}
```

```
{
  "locations": [[lng1, lat1], [lng2, lat2], ...],
  "metrics": ["distance", "duration"],
  "units": "km"
}
```


6.5.5. Fase 4 — Ordenación de la ruta con OR-Tools

La ordenación de paradas se modela con el *routing solver* de OR-Tools (Google OR-Tools Team, 2026) como un TSP de un solo vehículo cuyo depósito es la ubicación del usuario. El coste de cada arco (i, j) integra los tres criterios con los pesos del usuario:

$$c(i, j) = w_{\text{distancia}} \cdot \hat{d}_{ij} + w_{\text{tiempo}} \cdot \hat{t}_{ij} - w_{\text{precio}} \cdot \hat{s}_j \quad (6.1)$$

donde $\hat{d}_{ij}$ y $\hat{t}_{ij}$ son la distancia y el tiempo normalizados por el máximo de sus matrices, y $\hat{s}_j$ es el *ahorro normalizado* de la tienda destino j : la suma, sobre los ítems asignados a j , de la diferencia entre el siguiente mejor candidato y el precio elegido, dividida por el máximo ahorro entre tiendas. De este modo, un usuario que pondera el precio tolera desvíos hacia tiendas con mayor ahorro, mientras que uno que pondera distancia o tiempo obtiene rutas más compactas. El solver utiliza la estrategia `PATH_CHEAPEST_ARC` con un límite duro de 5 segundos y una dimensión de conteo que acota el número de paradas.

6.5.6. Fase 5 — Persistencia y desambiguación interactiva

El resultado se persiste de forma transaccional en `OptimizationResult`, `OptimizationRouteStop` y `OptimizationRouteStopItem`. Cuando la fase de resolución marca algún ítem como ambiguo, la respuesta incluye las alternativas semánticas; el endpoint `POST /api/v1/optimize/choices/` registra la elección del usuario como `ShoppingListSemanticPreference` y recalcula la ruta, de modo que las optimizaciones futuras de esa lista ya no repiten la pregunta.

6.5.7. Diagrama de secuencia del optimizador

La Figura 6.4 ilustra el flujo de mensajes entre componentes durante una petición de optimización.

6.5.8. Evolución respecto al diseño inicial

El planteamiento preliminar de la fase de análisis contemplaba enumerar combinaciones de tiendas y devolver las tres mejores rutas alternativas ordenadas por una función de score global. Durante la implementación (fase F5) ese diseño se sustituyó por el descrito en esta sección por dos razones contrastadas en las pruebas de aceptación: primera, la calidad percibida del resultado depende mucho más de resolver correctamente la ambigüedad ítem–producto (de ahí la capa semántica) que de explorar un gran número de combinaciones de tiendas; y segunda, una única ruta recomendada con desglose por parada y desambiguación interactiva resultó más accionable para el usuario que tres alternativas similares. La generación de rutas alternativas se mantiene como línea de trabajo futuro (Capítulo 9).

6.5.9. Complejidad y salvaguardas de rendimiento

El requisito RNF-001 (optimización en menos de 5 segundos) se garantiza *por construcción* mediante cotas explícitas en cada fase, resumidas en el Cuadro 6.4. No se ha realizado aún una campaña formal de medición de latencias (p95) sobre el conjunto de datos de demostración; queda documentada como trabajo pendiente en el Capítulo 8.

El endpoint se ejecuta de forma síncrona dentro del ciclo petición–respuesta: para los volúmenes del TFG (listas de decenas de ítems y decenas de tiendas por radio) las cotas anteriores mantienen la latencia dentro del objetivo. La ejecución asíncrona vía Celery y una caché de matrices de distancia en Redis se identificaron como optimizaciones de evolución para escenarios de mayor carga, pero no forman parte de la implementación actual.

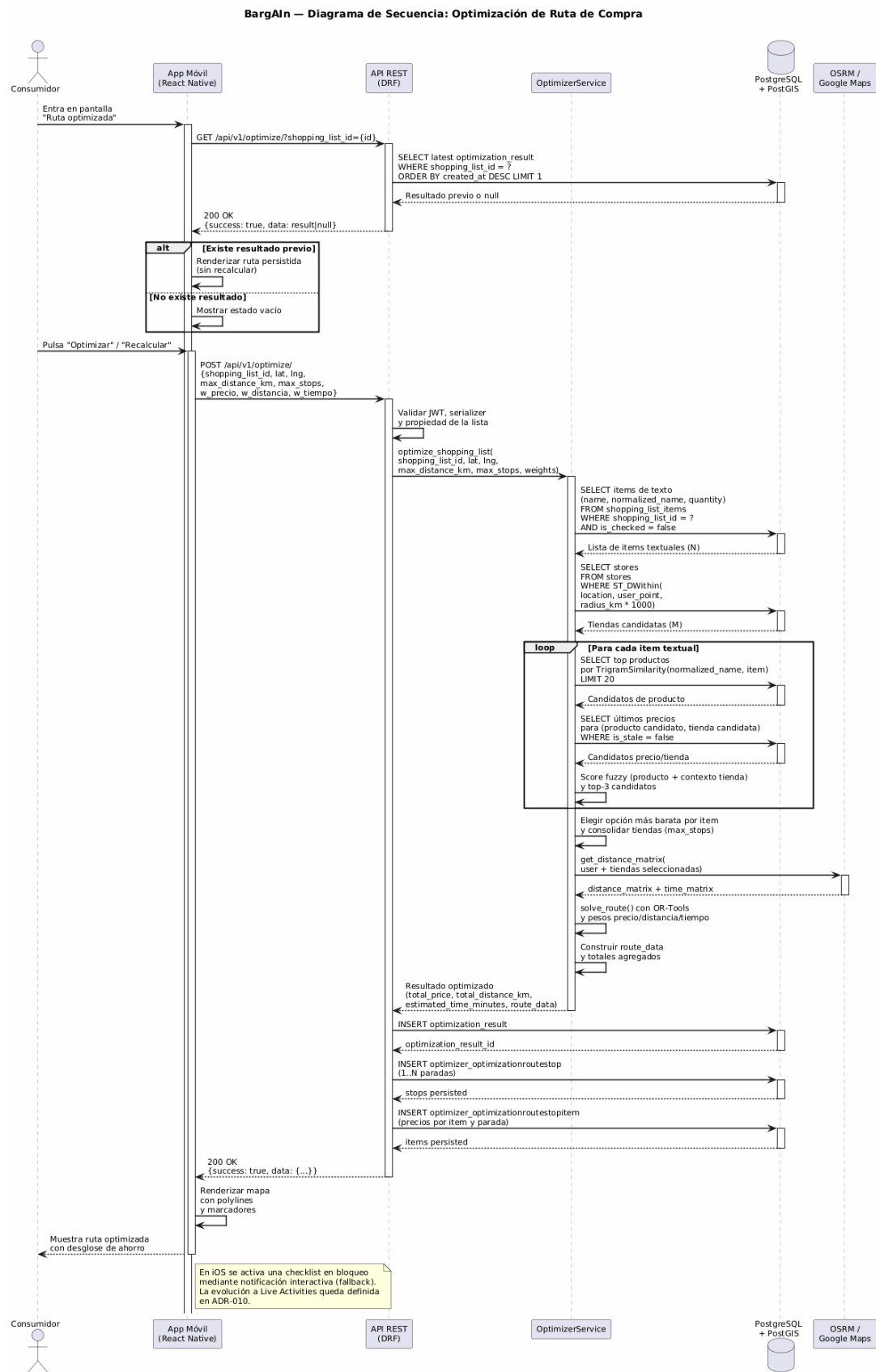


Figura 6.4: Diagrama de secuencia del proceso de optimización de ruta

Cuadro 6.4: Salvaguardas de rendimiento del optimizador

Fase	Mecanismo	Cota
Resolución de ítems	Índice GIN + límite de candidatos	20 productos por ítem
Puntuación de candidatos	Truncado del ranking	12 candidatos por ítem
Matriz de distancias	Petición única a ORS	1 llamada externa por optimización
Ordenación de ruta	<code>time_limit</code> del solver	≤ 5 s

6.6— Módulo de Web Scraping

El scraper es un **proyecto Scrapy independiente** (`scraping/`) (Scrapy Developers, 2026) desacoplado del código de la API. La persistencia se realiza directamente a través del ORM: el pipeline `PriceUpsertPipeline` inicializa el entorno Django (`django.setup()`) y escribe productos y precios normalizados en la base de datos, registrando la fuente como `scraping`.

El proyecto incluye **once spiders** (Mercadona, Carrefour, Lidl, DIA, Alcampo, Eroski, Hipercor, Consum, Covirán, Spar y Costco), todos cubiertos por tests unitarios. De ellos, **cuatro están en operación programada** mediante Celery Beat (Mercadona a las 06:00, Carrefour a las 06:30, Lidl a las 07:00 y DIA a las 07:30, una vez al día conforme a RN-010); los siete restantes están implementados y verificados a nivel de parser, pero no se han incorporado a la planificación periódica ni validado de forma continuada contra los sitios reales.

Cada cadena requiere una estrategia diferente:

- **Mercadona:** API interna (`tienda.mercadona.es/api/`) no oficial pero estable, accesible mediante peticiones HTTP estándar.
- **Carrefour / Lidl / DIA:** Catálogos con renderizado del lado del cliente (SPA), por lo que se usa **Playwright** (Microsoft Playwright Team, 2026) para ejecutar el JavaScript y obtener el DOM hidratado.

Todos los spiders declaran en sus `custom_settings` un `DOWNLOAD_DELAY` de 2 segundos entre peticiones consecutivas (regla de negocio RN-010).

6.6.1. Consideraciones legales y éticas del scraping

La obtención automatizada de precios públicos plantea cuestiones legales y éticas que el proyecto aborda explícitamente:

- **Naturaleza de los datos.** Los datos extraídos son precios de venta al público, información factual y pública sin datos personales; el RGPD (Parlamento Europeo y Consejo de la Union Europea, 2016) no aplica a esta extracción, aunque sí al tratamiento de los datos de los usuarios de BarGAIN (en particular su ubicación), que se realiza con consentimiento explícito.
- **Protocolo de exclusión.** La regla de negocio RN-009 exige respetar las directivas `robots.txt` (Koster, Illyes, Zeller, y Sassman, 2022) y excluir las cadenas que prohíban expresamente el acceso automatizado. Debe señalarse con honestidad que, en la configuración actual del proyecto Scrapy, la opción global `ROBOTSTXT_OBEY` está desactivada —los spiders operan contra rutas concretas con retardo de 2 segundos y frecuencia diaria—; su activación, junto con la revisión individual de las condiciones de uso de cada cadena, está identificada como mejora necesaria para alinear plenamente la implementación con RN-009 antes de cualquier operación a escala.

- **Minimización de impacto.** El retardo entre peticiones, la frecuencia máxima diaria por cadena y la preferencia por APIs públicas internas frente al renderizado completo reducen la carga impuesta a los servidores de destino, en línea con las obligaciones generales de diligencia de la LSSI-CE (Jefatura del Estado, 2002).
- **Estrategia de sustitución.** La prioridad de fuentes del sistema (API oficial > scraping > crowdsourcing, RN-004) permite reemplazar el scraping por acuerdos o APIs oficiales sin cambios de arquitectura, que es el escenario objetivo de una eventual explotación comercial.

6.7— Módulo OCR

El backend invoca **Google Cloud Vision API** (Google Cloud, 2026) con la imagen capturada por el usuario. La decisión de migrar desde Tesseract a Vision API (ADR-007) se adoptó tras comprobar que Tesseract no reconoce con suficiente claridad tickets y listas fotografiadas en condiciones reales. La integración se realiza contra la API REST de Vision con preprocesado previo de la imagen (fragmento simplificado de `apps/ocr/services.py`):

```
def extract_text_from_image(image_bytes: bytes,
                             lang: str = "spa+eng") -> list[str]:
    """Extrae líneas de texto con Google Vision y filtra ruido."""
    processed = _preprocess_image_for_ocr(image_bytes)
    response = _request_google_vision(processed, lang)
    raw_text = response.get("fullTextAnnotation", {}).get("text", "")
    return _extract_candidate_lines(raw_text)
```

Los códigos de idioma estilo Tesseract (`spa+eng`) se mantienen como alias de compatibilidad y se convierten internamente en *hints* BCP-47 para Vision. Cada línea extraída se normaliza y se empareja contra el catálogo con `thefuzz` (`token_sort_ratio`, umbral de confianza 80) (SeatGeek, 2026), devolviendo al cliente una lista de candidatos con su nivel de confianza para revisión manual.

6.7.1. Diagrama de secuencia OCR

La Figura 6.5 muestra el flujo de procesamiento desde la captura de imagen hasta la inserción de ítems en la lista.

6.8— Integración del Asistente LLM

El asistente conversacional utiliza la **Google Gemini API** (`gemini-2.0-flash-lite`) como modelo subyacente (ADR-008). El backend actúa como proxy, añadiendo contexto de la compra del usuario al system prompt antes de enviar cada mensaje a la API mediante el SDK `google-genai`.

Se aplica una **ventana deslizante** sobre el historial de mensajes (últimos 20 mensajes + system prompt) para mantener el consumo de tokens controlado. El system prompt incluye un guardrail que limita las respuestas a consultas de compra doméstica (RN-007), y el endpoint está protegido con un límite de 30 peticiones/hora por usuario.

Adicionalmente, la capa de resolución semántica del optimizador (Sección 6.5.2) emplea un modelo Gemini independiente y configurable mediante la variable `GEMINI_PRODUCT_MATCH_MODEL`, lo que permite usar un modelo distinto al del asistente conversacional para la clasificación de productos.

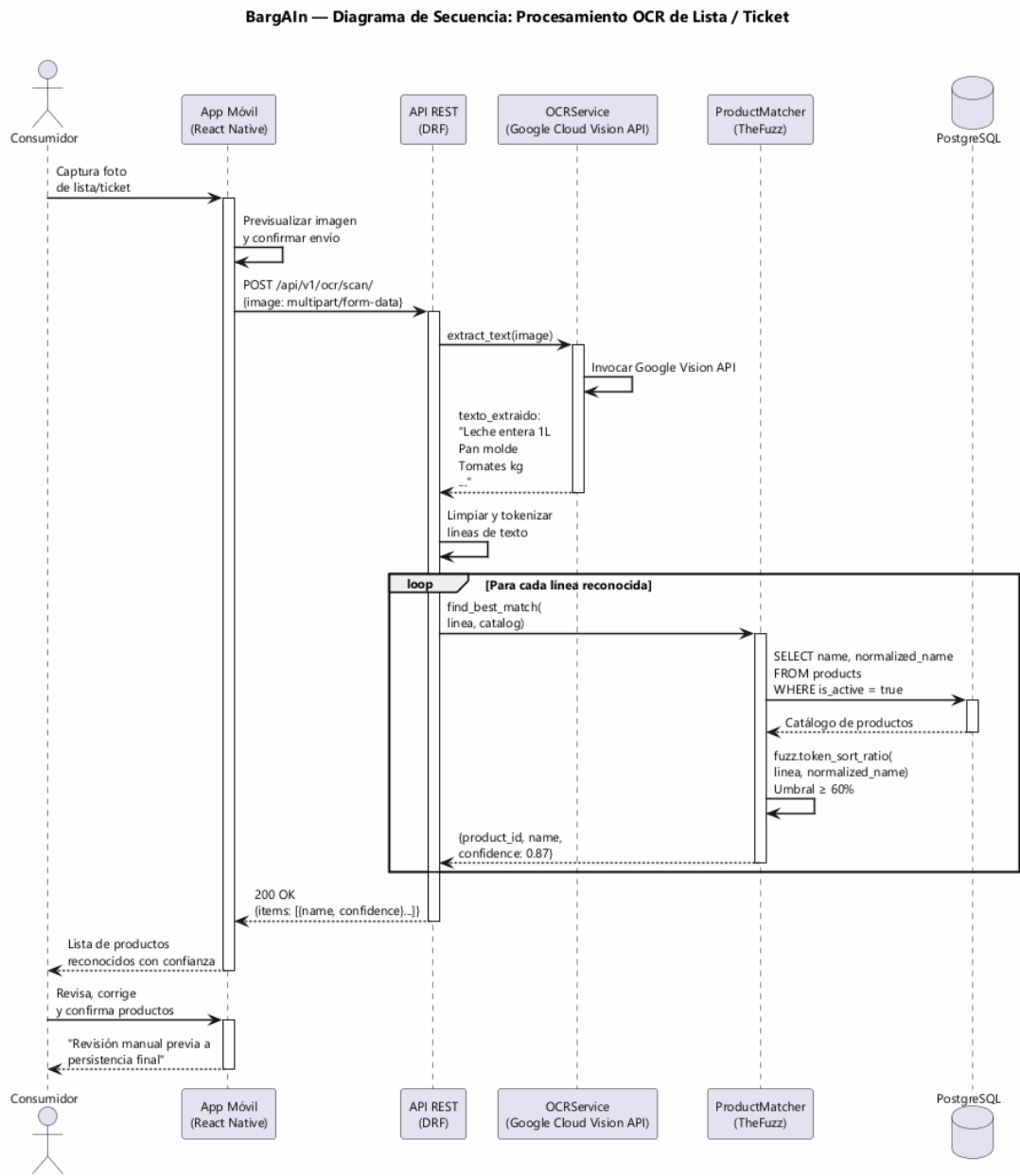


Figura 6.5: Diagrama de secuencia del flujo OCR

6.9— Infraestructura y Despliegue

6.9.1. Contenedores Docker

Cada componente se ejecuta en un contenedor independiente, orquestados con Docker Compose:

- **nginx:** Reverse proxy y servicio de estáticos.
- **backend:** Django + Gunicorn (3 workers).
- **celery-worker:** Workers para tareas asíncronas.
- **celery-beat:** Scheduler de tareas periódicas.
- **db:** PostgreSQL 16 + PostGIS.
- **redis:** Broker Celery + caché.

Para el entorno de desarrollo se usa el modelo híbrido (ADR-002): backend en Docker, frontend nativo en el host. Docker volumes en Windows rompen el HMR de Metro/Expo, por lo que el frontend no se dockeriza.

6.9.2. Pipeline CI/CD

El repositorio incluye cinco workflows de GitHub Actions:

- **ci-backend.yml:** Ejecuta `ruff check`, `ruff format --check` y `pytest --cov` (con umbral bloqueante `--cov-fail-under=80`) en cada push a cualquier rama.
- **ci-frontend.yml:** Ejecuta `eslint`, `prettier --check` y `jest --coverage` en cada push.
- **ios-build.yml:** Genera el `.ipa` unsigned mediante `expo prebuild + xcodebuild` en un runner `macos-latest`. El artefacto se descarga y se instala en el iPhone de demo con Sideloadly.
- **cd-render-staging.yml:** Despliega el backend en Render tras cada push a `main` que afecte a `backend/` (vía API de Render).
- **deploy-web-gh-pages.yml:** Compila el portal business (Vite) y lo publica en GitHub Pages.

El deploy a staging se realiza mediante **Render Blueprints**. El repositorio define dos variantes declarativas: `render.yaml`, con web service, worker Celery, scheduler beat, Redis y base de datos PostgreSQL independientes; y `render.free.yaml`, la variante free tier empleada durante el TFG, que consolida Gunicorn y un proceso Celery worker + beat embebido en un único web service. Cada push a `main` dispara un redesplicue automático.

6.9.3. Variables de entorno

La configuración sensible (claves API, credenciales de BD, secret key de Django) se gestiona exclusivamente mediante variables de entorno, nunca hardcodeadas. Se proporciona `.env.example` con los nombres de todas las variables requeridas.

6.10– Diseño de la Interfaz de Usuario

6.10.1. Sistema de diseño

El frontend define un sistema de diseño propio en `src/theme/`, bajo el concepto “Mercado Mediterráneo Digital”, inspirado en los mercados sevillanos. El Cuadro 6.5 recoge los tokens principales de color.

Cuadro 6.5: Paleta de colores del sistema de diseño (`src/theme/colors.ts`)

Token	Valor	Uso
<code>primary</code>	<code>#E8541A</code>	Naranja Triana: CTAs y énfasis de marca
<code>secondary</code>	<code>#2D5016</code>	Verde oliva: éxito, ahorro, acciones positivas
<code>accent</code>	<code>#F5C842</code>	Amarillo azulejo: acentos y badges
<code>background</code>	<code>#FAFAF7</code>	Fondo raíz de la aplicación
<code>surface</code>	<code>#FDF6EC</code>	Superficie de tarjetas y paneles
<code>success</code>	<code>#3A7D44</code>	Confirmaciones y precio reducido
<code>error</code>	<code>#C0392B</code>	Errores y avisos críticos
<code>text</code>	<code>#1A1A18</code>	Texto principal
<code>textMuted</code>	<code>#4F4F47</code>	Texto secundario

Tipografía: Fraunces para titulares y Source Sans 3 para texto (en iOS se delega en SF Pro, la tipografía del sistema), con IBM Plex Mono/Menlo para valores monoespaciados.
Espaciado: Escala de 4px.

6.10.2. Pantallas principales

La aplicación se estructura en cinco pestañas principales:

1. **Inicio / Dashboard:** Resumen de la lista activa, precio estimado en la tienda más barata cercana, acceso rápido al asistente y alertas activas.
2. **Mi Lista:** Gestión de la lista activa con buscador de autocompletado, grupos por categoría, swipe-to-delete/check y botón de optimizar ruta.
3. **Explorar:** Mapa interactivo con marcadores de tiendas y panel inferior deslizable con detalles de la tienda seleccionada.
4. **Optimizador:** Sliders de pesos, resumen agregado de la ruta recomendada con su desglose por parada, resolución de ambigüedades semánticas y visualización de la ruta en el mapa.
5. **Perfil:** Datos personales, preferencias de optimización, configuración de notificaciones y acceso al historial del asistente.

6.10.3. Adaptación a uso web (responsive y conveniencias de escritorio)

La misma base de código Expo (`frontend/src`) se reutiliza para ejecución en navegador de escritorio mediante `react-native-web`, sin duplicar pantallas ni alterar la navegación. La adaptación a escritorio se implementa como una capa transversal sobre las 15 pantallas existentes, organizada en cuatro ejes:

1. **Diseño responsive.** El hook `useBreakpoint()` (`src/hooks/`) clasifica el ancho del viewport en `mobile` (<768 px), `tablet` (768 – 1023 px) y `desktop` (≥ 1024 px). Las pantallas centran su contenido a un ancho máximo legible (entre 560 y 900 px según la pantalla) y los listados de tarjetas pasan de 1 columna en móvil a 2–4 columnas

en escritorio. El componente `MasterDetailLayout` aporta una disposición maestro-detalle de dos paneles, y la barra de pestañas inferior se centra a 900 px en escritorio.

2. **Interacción con ratón y teclado.** Se añaden estados *hover* (tintes y realces), atajos de teclado (`Cmd/Ctrl+K` para enfocar el buscador, `Enter` para añadir ítem, `Esc` para cerrar), *auto-focus* de campos al montar y *tooltips* web (`WebTooltip`). El reordenado de ítems de una lista emplea *drag-and-drop* HTML5 nativo en la variante `ListDetailScreen.web.tsx`.
3. **Conveniencias de escritorio.** Las utilidades de `src/utils/webExport.ts` permiten exportar la lista de la compra a `.txt`, la comparativa de precios a `.csv` y la ruta a fichero, así como copiar al portapapeles direcciones y enlaces. La generación de CSV incluye un *guard* de inyección de fórmulas (OWASP) que escapa las celdas que comienzan por `=`, `+`, `-` o `@`.
4. **Deep-linking.** La configuración `src/navigation/linking.ts` registra URLs estables para las pantallas (p.ej. `/app/lists/:listId`, `/app/map/store/:storeId`, `/app/home/catalog?q=...`), de modo que el estado de la aplicación es enlazable y navegable desde la barra de direcciones del navegador.

Todas las adiciones están condicionadas a la plataforma web (comprobación de `Platform.OS`) o al *breakpoint* activo, de forma que el comportamiento en móvil permanece inalterado. Estas mejoras se incorporaron en una fase específica de adaptación web posterior al cierre del plan inicial, y se validaron con 111 pruebas Jest (15 *suites*) y una batería de verificación manual en navegador.

Manual de Usuario

7.1— Introducción

Este capítulo describe, desde la perspectiva del usuario final, el funcionamiento completo de BarGAIN en sus tres contextos de uso: la aplicación móvil para consumidores (iOS), su versión de escritorio ejecutada en navegador, y el portal web orientado a pequeños y medianos comercios (PYMEs). La exposición sigue el recorrido natural de un usuario: desde el registro y el acceso, pasando por la gestión de listas de la compra y la comparación de precios, hasta las funcionalidades diferenciales del sistema —la optimización multicriterio de rutas de compra, el reconocimiento óptico de tickets y el asistente conversacional—, para concluir con las herramientas de gestión que ofrece el portal *business*.

Todas las imágenes que ilustran este capítulo son capturas reales de la aplicación desplegada en el entorno de desarrollo descrito en el Capítulo 6, obtenidas sobre un conjunto de datos de demostración representativo (catálogo normalizado, precios de varias cadenas de supermercados en Sevilla y un comercio local verificado). Las capturas de la aplicación móvil corresponden a un *viewport* de 390×844 puntos, equivalente al de un teléfono actual de gama media; las de escritorio, a una ventana de 1440×900.

7.2— Despliegue y acceso al sistema

BarGAIN se encuentra desplegado en infraestructura pública: el backend (API REST, base de datos PostgreSQL/PostGIS y cola Redis) se ejecuta en *Render* bajo la URL <https://bargain-free-api.onrender.com>, y el portal para comercios se sirve como aplicación estática desde *GitHub Pages* en <https://qh0329.github.io/bargain-tfg/>. La aplicación de usuario final, construida con Expo, puede ejecutarse en un navegador de escritorio desde el propio repositorio o instalarse en un iPhone a partir del artefacto generado por la integración continua. Todas las variantes del cliente consumen el mismo backend de *Render*, por lo que los datos son consistentes entre plataformas.

Conviene tener presente una particularidad del plan gratuito de *Render*: tras un periodo de inactividad el servicio se hiberna, de modo que la primera petición puede demorarse en torno a un minuto (*cold start*) mientras la instancia se reactiva. Las peticiones posteriores responden con normalidad.

7.2.1. Acceso al portal para empresas (GitHub Pages)

El portal *business* no requiere instalación alguna: basta un navegador moderno. El procedimiento de acceso es el siguiente:

1. Navegar a <https://qh0329.github.io/bargain-tfg/>. Se carga la página de presentación del portal.
2. Pulsar *Empezar onboarding* (o navegar a `/login` si ya se dispone de cuenta).
3. Registrar la cuenta del comercio con sus datos de acceso (usuario, correo y contraseña) y los datos del negocio (razón social, CIF/NIF y dirección).
4. Esperar la verificación del perfil por parte de un administrador; hasta entonces la cuenta permanece en estado *pendiente*.
5. Una vez aprobado, iniciar sesión para acceder al centro de operaciones descrito en la Sección 7.15.

7.2.2. Ejecución de la aplicación de usuario en local (Expo web)

La aplicación de consumo puede ejecutarse en cualquier equipo de escritorio sin necesidad de dispositivo móvil, reutilizando el soporte web de Expo. Los únicos prerequisites son *Git* y *Node.js* 24 o superior (que incluye *npm*). El procedimiento es:

1. Clonar el repositorio y situarse en el directorio del frontend:

```
git clone https://github.com/QHX0329/bargain-tfg.git
cd bargain-tfg/frontend
```

2. Instalar las dependencias del proyecto:

```
npm install
```

3. Revisar la configuración de entorno del fichero `frontend/.env.local`. La variable `EXPO_PUBLIC_API_URL` apunta por defecto al backend desplegado en Render:

```
EXPO_PUBLIC_API_URL=https://bargain-free-api.onrender.com/api/v1
```

Para trabajar contra un backend local en Docker basta sustituirla por `http://localhost:8000/api/v1`

4. Arrancar el servidor de desarrollo en modo web:

```
npx expo start --web
```

5. Abrir `http://localhost:8081` en el navegador. Se mostrará la pantalla de inicio de sesión; el registro de una cuenta nueva se realiza desde la propia interfaz.

7.2.3. Instalación de la aplicación en iPhone

La distribución para iOS se apoya en el flujo de integración continua *iOS Build* (GitHub Actions), que compila la aplicación en un runner macOS y publica como artefacto una IPA sin firmar (*BarGAIN-unsigned*). Al no estar distribuida a través del App Store, la instalación requiere firmar la IPA con un identificador de Apple, operación que resuelven herramientas de *sideloading* como *Sideloadly* o *AltStore*. El procedimiento con Sideloadly es el siguiente:

1. Descargar el artefacto *BarGAIN-unsigned* de la última ejecución del workflow *iOS Build* en la pestaña *Actions* del repositorio, y descomprimirlo para obtener el fichero `.ipa`.

2. Conectar el iPhone al ordenador mediante USB y autorizar el equipo desde el teléfono.
3. Abrir Sideloadly, arrastrar la IPA a la ventana de la aplicación, seleccionar el dispositivo e introducir un Apple ID. Con una cuenta gratuita, la firma personal tiene una validez de siete días, transcurridos los cuales basta repetir la operación; con una cuenta del programa de desarrolladores la validez se extiende a un año.
4. Pulsar *Start* y esperar a que finalice la firma e instalación.
5. En el iPhone, confiar en el certificado del desarrollador desde *Ajustes* → *General* → *VPN y gestión de dispositivos*.
6. Abrir BarGAIN. La aplicación se conecta automáticamente al backend de Render, por lo que no requiere configuración adicional; conceder el permiso de ubicación cuando se solicite para habilitar el mapa y la optimización de rutas.

La versión actual de BarGAIN se distribuye exclusivamente para iPhone (iOS 14 o superior) y navegadores de escritorio; las funciones de mapa y optimización requieren además el permiso de ubicación concedido y conexión a Internet.

Para el entorno de desarrollo local completo (backend incluido), el sistema se levanta con el backend en Docker (`make dev`), la base de datos migrada (`make migrate`) y el frontend Expo ejecutado de forma nativa en el host (`make frontend`), conforme al modelo híbrido adoptado en el ADR-002.

7.3— Registro y acceso al sistema

El acceso a la aplicación se articula en torno a un esquema clásico de autenticación con nombre de usuario y contraseña sobre tokens JWT. La pantalla de inicio de sesión (Figura 7.1, izquierda) presenta un formulario deliberadamente austero —dos campos y una única acción principal— con el fin de minimizar la fricción de entrada. El formulario de registro (Figura 7.1, derecha) solicita únicamente los datos imprescindibles para crear la cuenta: nombre de usuario, nombre y apellidos, correo electrónico (único, empleado para comunicaciones y recuperación de contraseña) y contraseña con confirmación, aplicando validación inmediata en el cliente antes de delegar en la validación definitiva del servidor.

Tras una autenticación satisfactoria, el sistema emite un par de tokens (acceso y refresco) y carga la navegación principal por pestañas. La sesión se mantiene de forma transparente: si el token de acceso expira, el interceptor HTTP lo renueva automáticamente con el token de refresco, de modo que el usuario solo vuelve a introducir credenciales si permanece inactivo más allá de la vida útil de este último. En el cliente móvil ambos tokens se custodian en el almacenamiento seguro del sistema operativo (*Keychain* de iOS).

7.4— Pantalla principal

La pantalla de inicio (Figura 7.2) actúa como cuadro de mando del usuario y condensa, en un único recorrido vertical, la información de mayor valor inmediato: un buscador global con acceso directo al catálogo; una botonera de acciones rápidas (plantillas, productos, escaneo OCR y favoritos); las notificaciones más recientes; las listas activas con su número de productos; las tiendas detectadas en el radio configurado; y las alertas de precio vigentes con su precio objetivo. Esta disposición responde a un principio de diseño que ha guiado toda la interfaz: las tareas frecuentes deben resolverse en un máximo de tres interacciones desde el arranque de la aplicación.

The image displays two side-by-side screenshots of the BarGAIN application's user interface. The left screenshot shows the login page, featuring the BarGAIN logo at the top, the tagline 'Tu compra inteligente', and a login form with fields for 'Usuario' (containing 'demo@bargain.local') and 'Contraseña' (masked with dots). Below the form is an orange 'Iniciar sesión' button, and at the bottom are links for '¿Olvidaste tu contraseña?' and '¿No tienes cuenta? Regístrate'. The right screenshot shows the registration page, titled 'Crea tu cuenta' with the subtitle 'Únete a BargAln y empieza a ahorrar'. It includes a form with fields for 'Nombre de usuario' (Nico), 'Nombre' (Parrilla), 'Apellidos' (nico@example.), 'Email' (Demo1234!), 'Contraseña' (masked), and 'Repetir' (Repetir). An orange 'Crear cuenta' button is at the bottom, along with a link '¿Ya tienes cuenta? Inicia sesión'.

Figura 7.1: Acceso al sistema: inicio de sesión (izquierda) y registro de un nuevo usuario (derecha)

7.5— Gestión de listas de la compra

La lista de la compra es la entidad vertebradora de BarGAIN: sobre ella se calculan totales por tienda, se lanza la optimización de rutas y se articula la colaboración entre usuarios. La pestaña *Listas* (Figura 7.3, izquierda) muestra las listas del usuario con acciones directas de duplicado, conversión en plantilla y archivado. El detalle de una lista (Figura 7.3, derecha) presenta los ítems con controles de cantidad “+/-”, casillas de verificación para marcar productos como comprados y acciones de exportación y compartición en la cabecera.

El flujo de trabajo habitual es el siguiente: el usuario crea una lista con nombre personalizado; añade productos desde el catálogo, desde el buscador con autocompletado o mediante el escáner OCR; ajusta cantidades; y, durante la compra, marca cada producto adquirido. Una lista puede compartirse con hasta diez colaboradores, cuyas modificaciones se notifican al resto de participantes, y puede convertirse en plantilla para institucionalizar compras recurrentes. Las plantillas (Figura 7.4) funcionan como generadores de listas: a partir de una plantilla —por ejemplo, la compra básica semanal— se instancian nuevas listas con un solo gesto, evitando reconstruir manualmente colecciones de productos que apenas varían de una semana a otra.

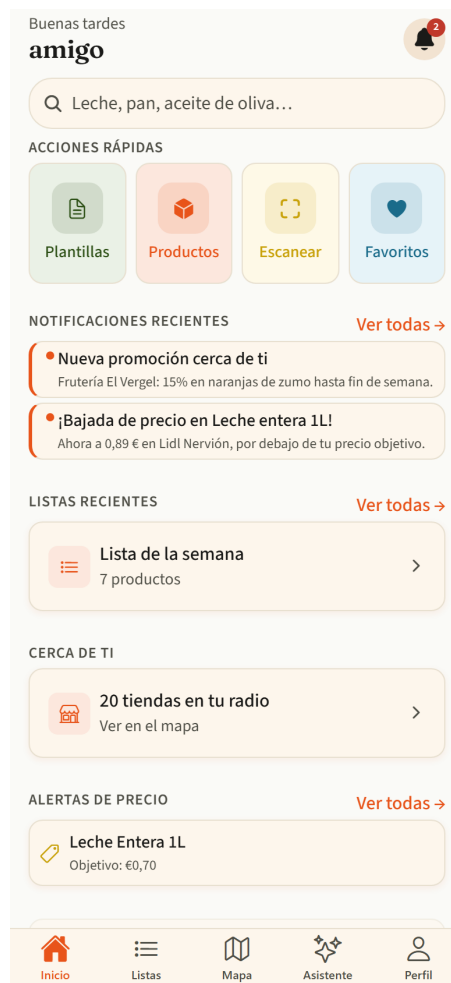


Figura 7.2: Pantalla principal de la aplicación móvil, con notificaciones, listas activas, tiendas cercanas y alertas de precio

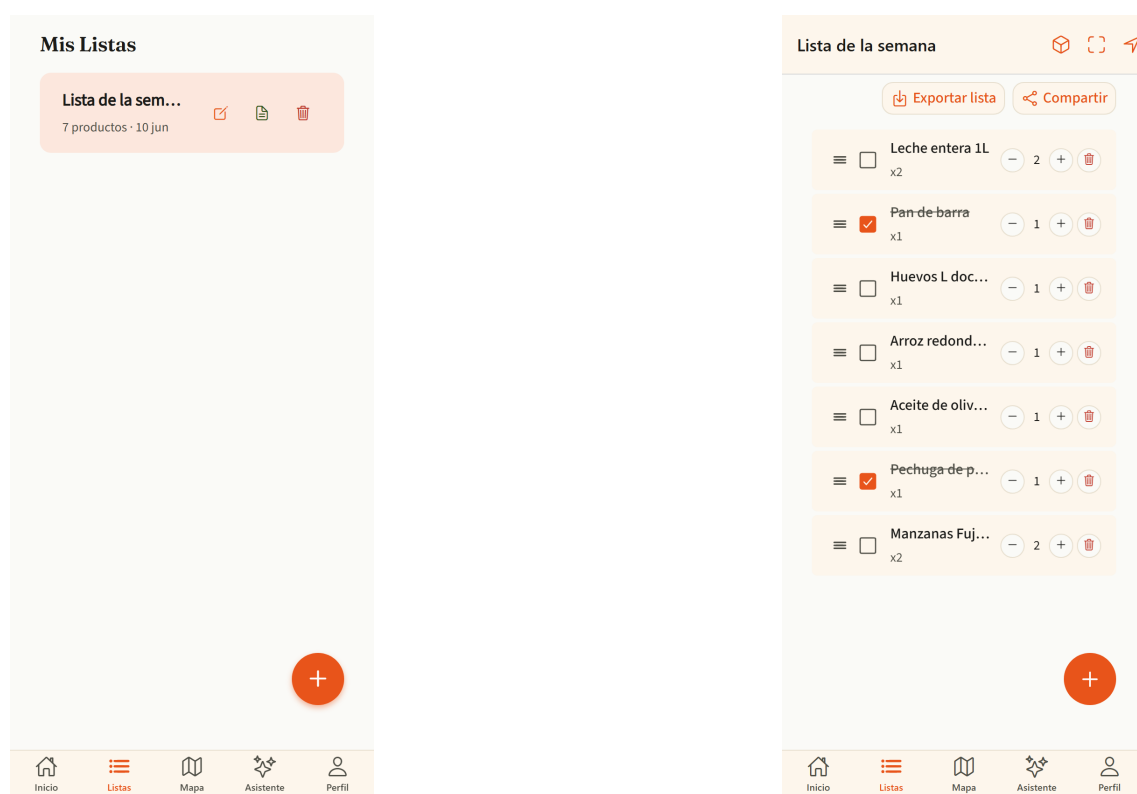


Figura 7.3: Gestión de listas: colección de listas del usuario (izquierda) y detalle de una lista con controles de cantidad y verificación (derecha)

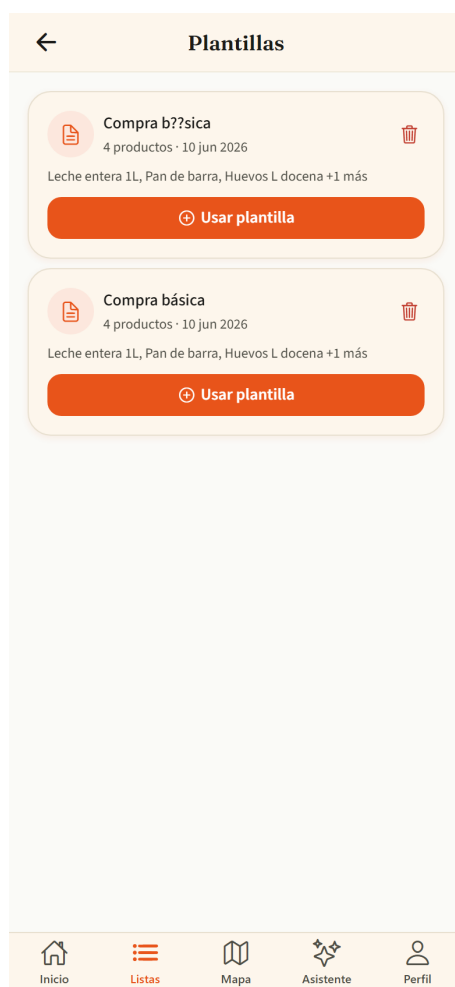


Figura 7.4: Plantillas de lista: instanciación de compras recurrentes a partir de una plantilla guardada

7.6— Catálogo y comparativa de precios

El catálogo (Figura 7.5, izquierda) ofrece una vista paginada del repositorio de productos normalizados, con filtros plegables por categoría, precio y distancia. Cada tarjeta sintetiza la información decisiva para el usuario: el precio mínimo detectado y la tienda que lo ofrece, junto con una acción de añadido rápido a la lista activa que se resuelve con un único toque.

Desde cualquier tarjeta se accede a la comparativa de precios del producto (Figura 7.5, derecha), que constituye una de las aportaciones centrales del sistema: una tabla ordenable con el precio del artículo en cada tienda del radio configurado, señalando las ofertas vigentes y la antigüedad de cada dato según su fuente (API oficial, *scraping* o aportación de usuarios, por este orden de prioridad). La misma pantalla incorpora el histórico de precios con un gráfico de evolución temporal, que permite valorar si el precio actual es coyuntural o estable, y la creación de alertas de precio sobre el producto consultado.

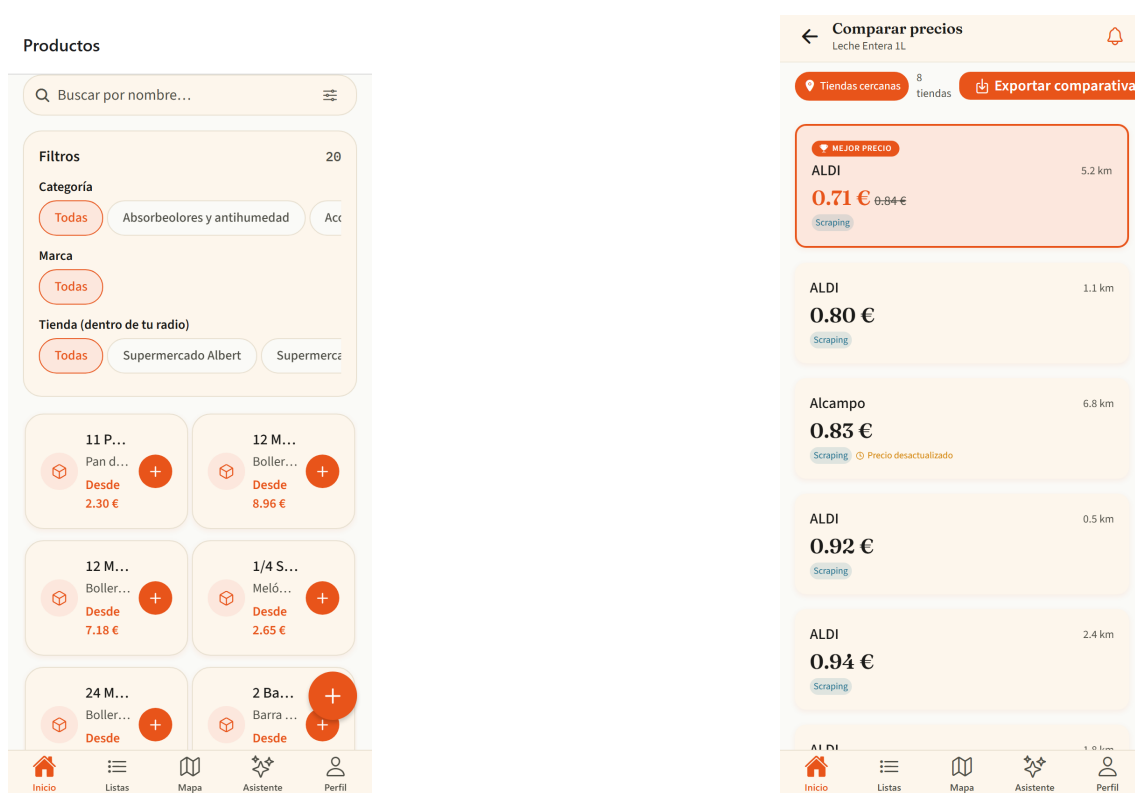


Figura 7.5: Catálogo de productos con filtros y añadido rápido (izquierda) y comparativa de precios por tienda de un producto (derecha)

El catálogo es deliberadamente abierto a la participación: cuando un usuario no encuentra un producto, puede proponer su alta mediante el formulario de la Figura 7.6, indicando nombre, marca, código de barras y categoría. La propuesta queda pendiente de revisión por un administrador, lo que preserva la calidad del repositorio común sin renunciar al crecimiento colaborativo del mismo.

Proponer producto

INFORMACIÓN DEL PRODUCTO

Nombre *

Ej. Leche semidesnatada 1L

Marca

Ej. Hacendado

Código EAN-13

Ej. 8412345678901

PRECIO

Precio (€)

0.00

Precio unitario (€/kg o €/L)

Ej. 1.20

Tienda

Supermercado Albert Supermercados El Jamón

Notas adicionales

Ej. Encontré este producto en el supermercado X pero no aparece en la app

Inicio Listas Mapa Asistente Perfil

Figura 7.6: Propuesta de alta de un nuevo producto en el catálogo, sujeta a aprobación por el administrador

7.7– Mapa de tiendas

La pestaña *Mapa* (Figura 7.7, izquierda) sitúa al usuario en el centro de su entorno comercial: sobre cartografía interactiva se representan las tiendas del radio configurado con marcadores diferenciados por cadena, y un panel inferior plegable lista los establecimientos visibles con su distancia y tiempo estimado de desplazamiento. Al seleccionar un establecimiento se abre su perfil (Figura 7.7, derecha), que reúne la información de contacto y horario junto con el catálogo de productos detectados en esa tienda y sus precios, de modo que el usuario puede evaluar un comercio concreto sin abandonar el contexto del mapa.

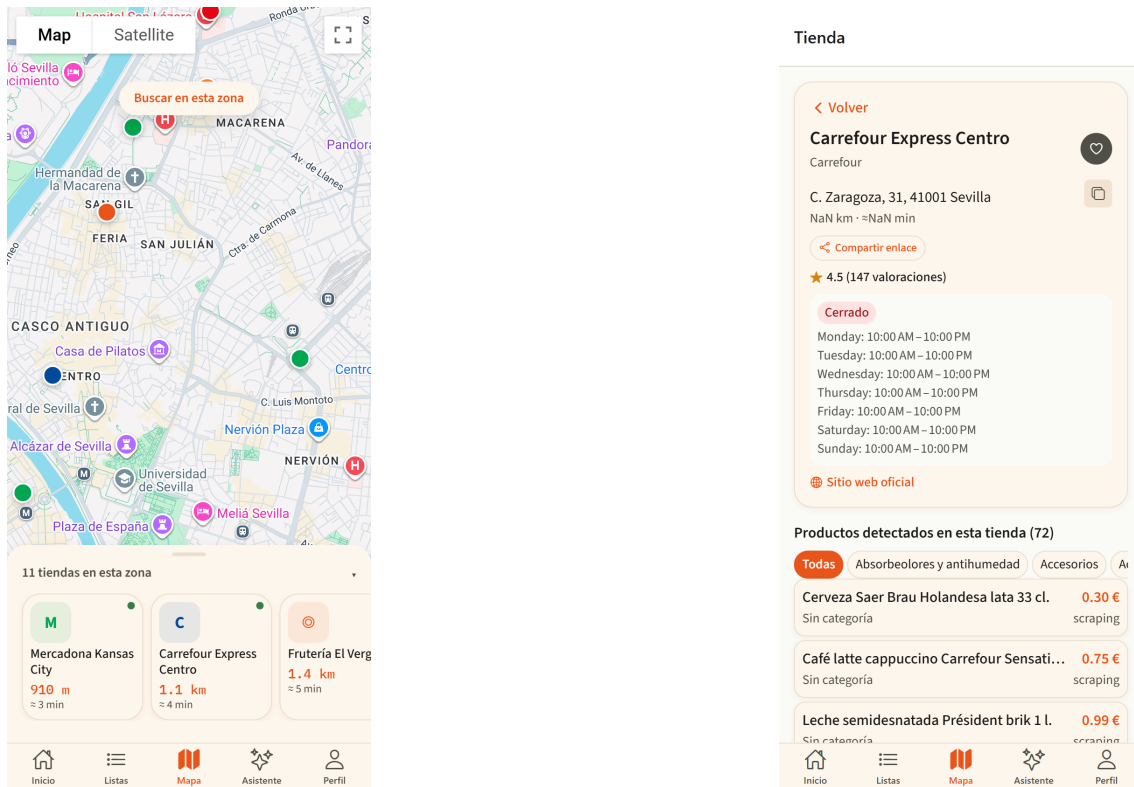


Figura 7.7: Mapa de tiendas con marcadores por cadena y panel de establecimientos (izquierda) y perfil de tienda con sus productos y precios (derecha)

El sistema permite marcar tiendas como favoritas, que se agrupan en una vista propia (Figura 7.8) accesible tanto desde el mapa como desde las acciones rápidas de la pantalla principal. Las tiendas favoritas reciben un tratamiento preferente en la generación de candidatos del optimizador de rutas, reflejando la fidelidad del usuario a determinados establecimientos.

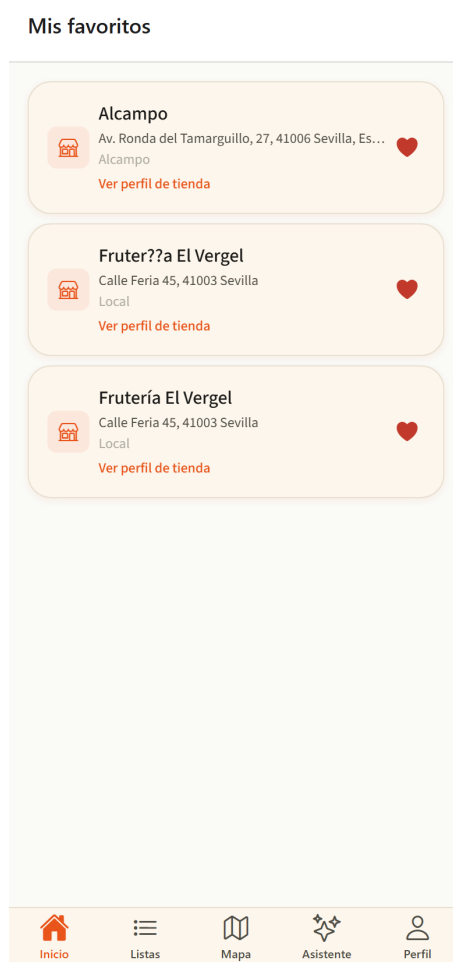


Figura 7.8: Tiendas favoritas del usuario, con acceso directo a su perfil

7.8— Alertas de precio y notificaciones

BarGAIN invierte la carga de la vigilancia de precios: en lugar de obligar al usuario a consultar periódicamente la comparativa, es el sistema quien le avisa cuando se cumple una condición de interés. La bandeja de notificaciones (Figura 7.9, izquierda) centraliza tres familias de avisos —bajadas de precio que alcanzan el objetivo fijado, nuevas promociones de comercios cercanos y cambios en listas compartidas— con marcado de leídas individual o masivo, borrado lógico y enlaces profundos que conducen directamente a la pantalla afectada. Estos mismos avisos se entregan como notificaciones *push* en el dispositivo.

La gestión de alertas (Figura 7.9, derecha) muestra las alertas activas con su precio objetivo y permite editarlas o eliminarlas. Una alerta se crea desde la comparativa de un producto indicando el umbral deseado; cuando algún establecimiento publica un precio igual o inferior, el backend dispara la notificación correspondiente.

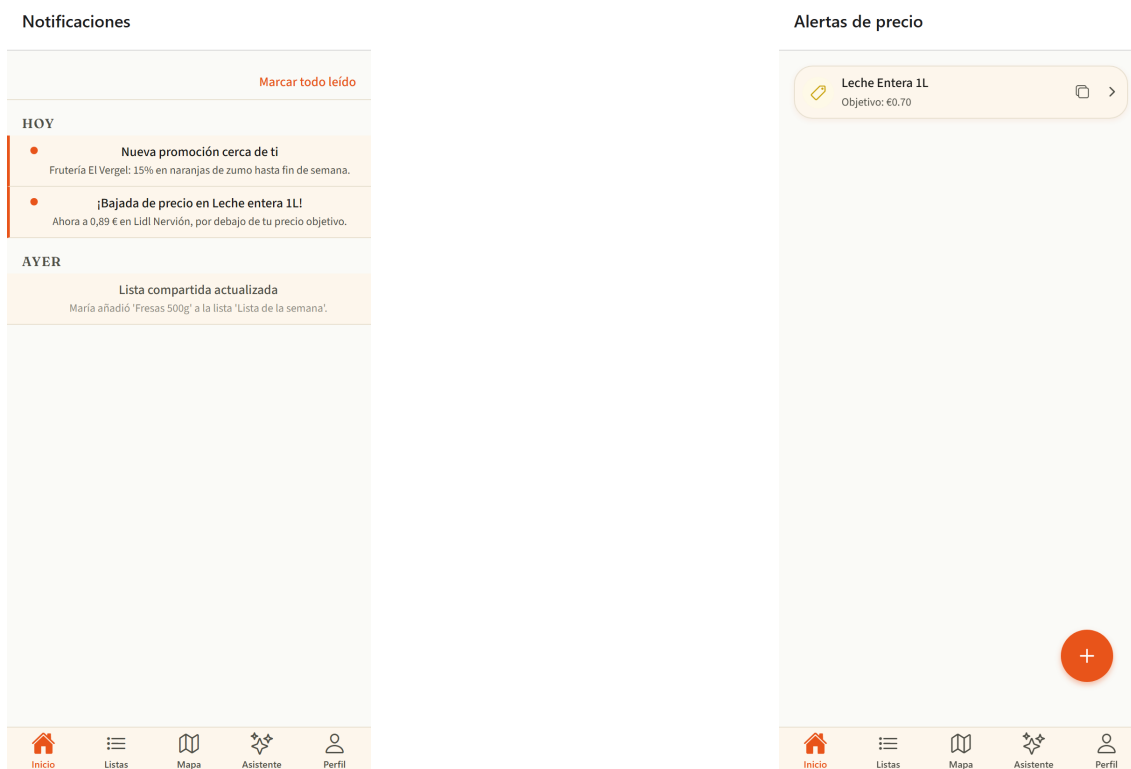


Figura 7.9: Bandeja de notificaciones con avisos de precio, promociones y listas compartidas (izquierda) y gestión de alertas de precio con su objetivo (derecha)

7.9— Perfil y preferencias del usuario

La pestaña *Perfil* (Figura 7.10, izquierda) agrupa la identidad del usuario y la configuración global de la aplicación: datos personales, preferencias de notificación y cierre de sesión, que elimina ambos tokens del almacenamiento seguro del dispositivo (el token de refresco rotado queda además en lista negra en el servidor en la siguiente renovación). Su apartado más relevante para el comportamiento del sistema es la configuración del optimizador (Figura 7.10, derecha), donde el usuario gobierna la función de *scoring* multicriterio descrita en el Capítulo 6: el radio máximo de búsqueda (1–20 km), el número máximo de paradas por ruta y, sobre todo, los pesos relativos w_{precio} , $w_{distancia}$ y w_{tiempo} , ajustables mediante deslizadores cuya suma se normaliza automáticamente a la unidad. De este modo, una misma lista produce rutas distintas para un usuario que prioriza el ahorro

frente a otro que prioriza la inmediatez, sin que ninguno de los dos necesite comprender el algoritmo subyacente.

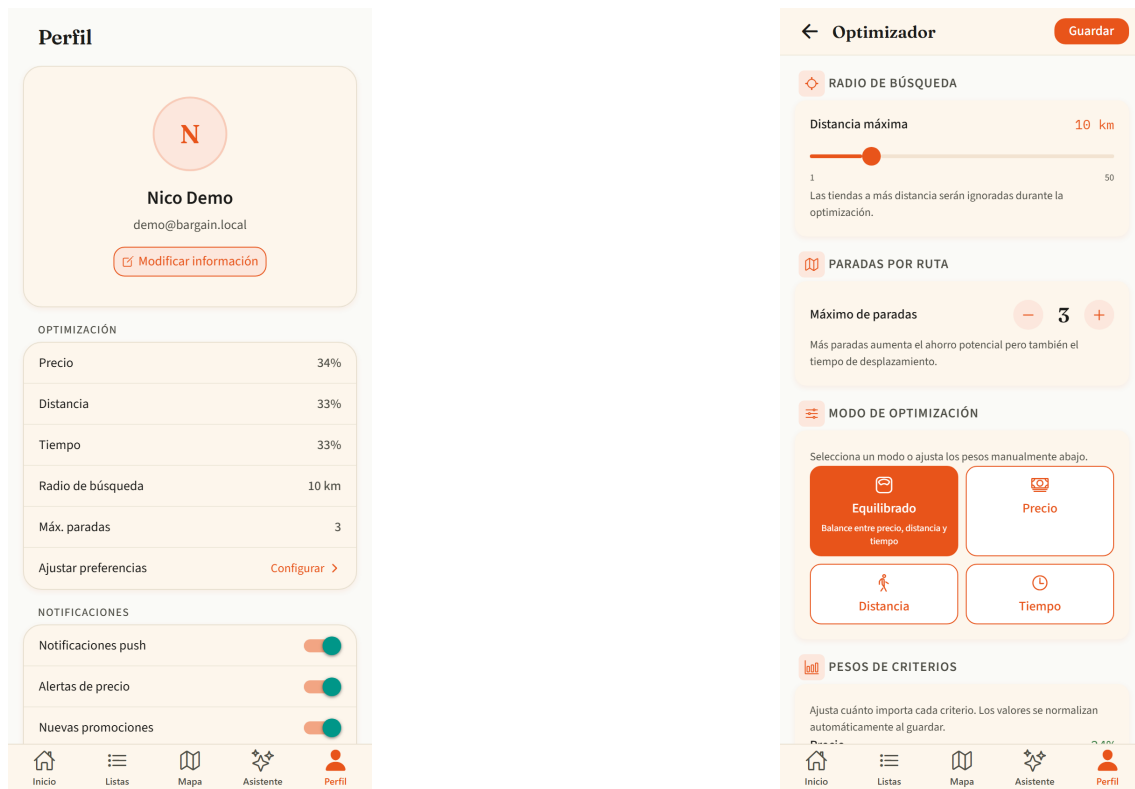


Figura 7.10: Perfil del usuario (izquierda) y configuración del optimizador con radio, paradas y pesos del *scoring* multicriterio (derecha)

Las operaciones sobre la cuenta se completan con la edición de los datos básicos y el cambio de contraseña (Figura 7.11), este último con verificación de la contraseña vigente y validación de robustez de la nueva.

Modificar información

Modificar información

Actualiza tus datos personales para mantener tu perfil al día.



Toca para cambiar la foto

Nombre

Apellidos

Email

Guardar cambios

Cambiar contraseña

Cambiar contraseña

Introduce tu contraseña actual y elige una nueva de al menos 8 caracteres.

Contraseña actual

Nueva contraseña

Confirmar contraseña

Guardar contraseña

Figura 7.11: Operaciones sobre la cuenta: edición del perfil (izquierda) y cambio de contraseña (derecha)

7.10– Ruta de compra optimizada

La optimización de rutas es la funcionalidad nuclear del proyecto y el punto donde convergen precios, geolocalización y preferencias del usuario. Desde el detalle de una lista, la acción *Ruta optimizada* conduce a la pantalla de la Figura 7.12, que resume el resultado de la última optimización persistida y permite recalcularla con la ubicación y los pesos actuales.

La cabecera recoge los parámetros empleados (radio y pesos), seguida del selector de paradas máximas (2–5) y del botón de recálculo. El bloque central presenta el resultado agregado —precio total estimado, distancia del recorrido, duración aproximada y número de paradas— y, a continuación, el desglose por parada: cada establecimiento de la ruta, con su distancia y tiempo parciales, enumera los productos que conviene adquirir en él junto con su precio. En el ejemplo de la figura, el optimizador ha asignado las manzanas a un comercio local porque su precio compensa el desvío conforme a los pesos configurados.

Cuando un ítem de la lista admite varias interpretaciones en el catálogo, la capa semántica respaldada por el modelo Gemini señala la ambigüedad y ofrece las alternativas (“Manzanas Fuji”, “Manzanas Golden”...), cada una con la acción *usar y recalcular*; la elección se persiste como preferencia de esa lista, de modo que las optimizaciones futuras ya no repiten la pregunta. El comportamiento de la pantalla es persistente: al salir y volver, se recupera la última ruta calculada desde la base de datos, y *Ver en mapa* abre el recorrido completo en la aplicación de mapas del dispositivo.



Figura 7.12: Ruta de compra optimizada: parámetros, resultado agregado, desglose por parada y resolución de ambigüedades semánticas con recálculo

7.11– Reconocimiento de tickets y listas (OCR)

El módulo OCR reduce drásticamente el coste de entrada de datos: en lugar de teclear una lista, el usuario fotografía un ticket de compra o una lista manuscrita y el sistema la convierte en ítems estructurados. El flujo sigue un asistente de tres pasos. En el primero (Figura 7.13, izquierda), la pantalla de captura ofrece la cámara o la galería como origen de la imagen. El segundo paso (Figura 7.13, derecha) presenta el resultado del reconocimiento —delegado en Google Cloud Vision y refinado con un emparejamiento difuso contra el catálogo—: cada línea detectada aparece con el texto original, el producto del catálogo con el que se ha emparejado y una etiqueta de confianza codificada por color (verde a partir del 80 %, amarillo entre el 50 y el 79 %, rojo por debajo). El usuario actúa como verificador final: corrige nombres en línea, ajusta cantidades con los selectores y descarta las líneas irrelevantes. En el tercer paso confirma la operación, eligiendo entre volcar los ítems a una lista existente o crear una nueva.

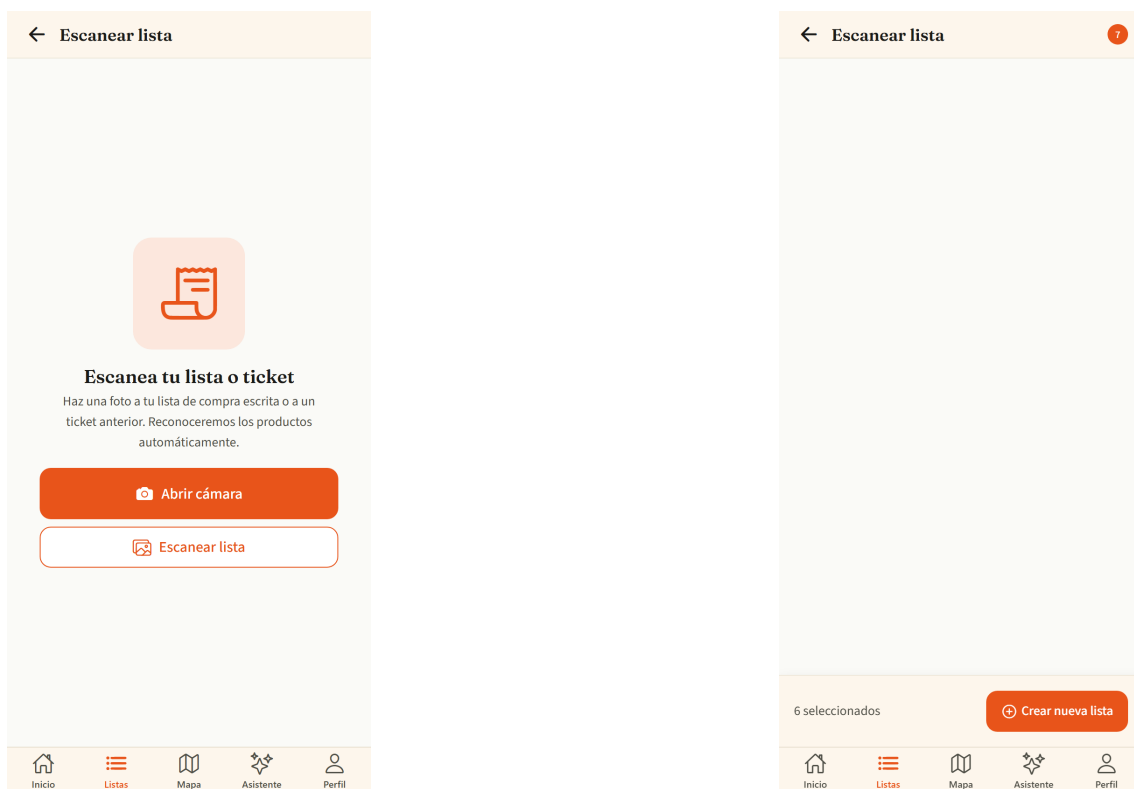


Figura 7.13: Asistente OCR: captura del ticket o lista (izquierda) y revisión de los ítems reconocidos con su nivel de confianza (derecha)

7.12– Asistente conversacional

El asistente (Figura 7.14) incorpora un modelo de lenguaje (Gemini, mediado por el backend conforme al ADR-008) especializado en la compra doméstica. La pantalla sigue el patrón conversacional estándar —mensajes del usuario a la derecha, respuestas del asistente a la izquierda— e incluye sugerencias de arranque que comunican el alcance del sistema: comparar precios entre cadenas, localizar la tienda más barata para un producto o pedir la optimización de la lista activa. Cuando una respuesta menciona productos, precios o tiendas, incorpora fichas interactivas que permiten añadir el producto a la lista sin abandonar la conversación. El asistente está acotado deliberadamente a su dominio:

ante preguntas ajenas a la compra responde declinando la consulta, como parte de las salvaguardas de uso descritas en el Capítulo 6.

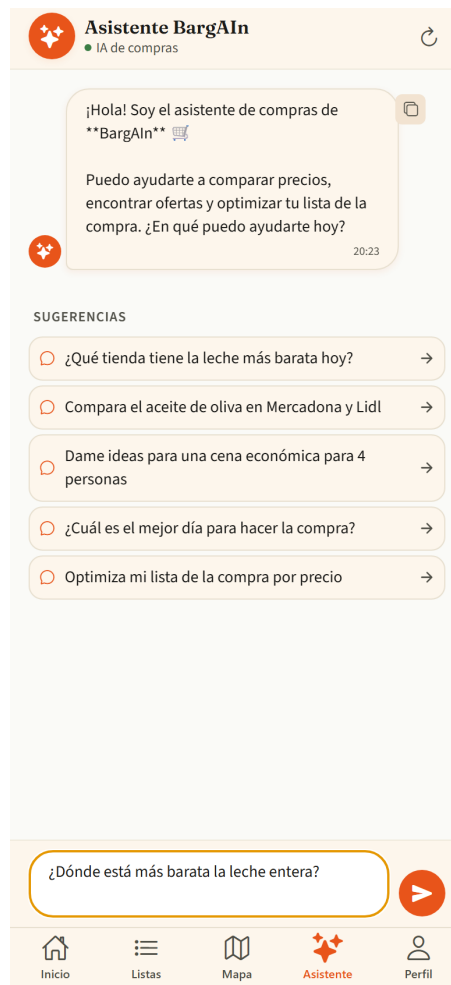


Figura 7.14: Asistente conversacional con sugerencias de inicio y campo de consulta; las respuestas con productos incluyen fichas de añadido directo a la lista

7.13— Lista de comprobación en pantalla de bloqueo (iOS)

Como complemento del flujo de compra presencial, desde la pantalla de ruta optimizada el botón *Checklist en bloqueo* publica una notificación interactiva con los productos de la parada actual. El usuario puede marcar cada artículo como comprado directamente desde la pantalla bloqueada del teléfono, sin desbloquear el dispositivo ni reabrir la aplicación; la notificación se actualiza tras cada acción. La versión basada en *Live Activities* de ActivityKit queda fuera del flujo gestionado de Expo y se documenta como trabajo futuro en el ADR-010.

7.14— La aplicación en el navegador de escritorio

La misma base de código Expo de la aplicación móvil se ejecuta en navegadores de escritorio, donde la interfaz se reorganiza para aprovechar la superficie adicional y los dispositivos de entrada propios del ordenador. Esta sección documenta dicha adaptación, que no es una mera ampliación del lienzo: cambia la disposición, la densidad de información y las *affordances* de interacción.

La pantalla principal (Figura 7.15, izquierda) centra el contenido a un ancho legible y convierte las acciones rápidas en una botonera horizontal. El catálogo (Figura 7.15, derecha) pasa de la lista vertical móvil a una rejilla de dos a cuatro columnas según el ancho de la ventana, con realces *hover* sobre las tarjetas; el atajo **Cmd/Ctrl+K** enfoca el buscador desde cualquier punto, **Enter** añade el producto resaltado a la lista y **Esc** cierra los diálogos abiertos.

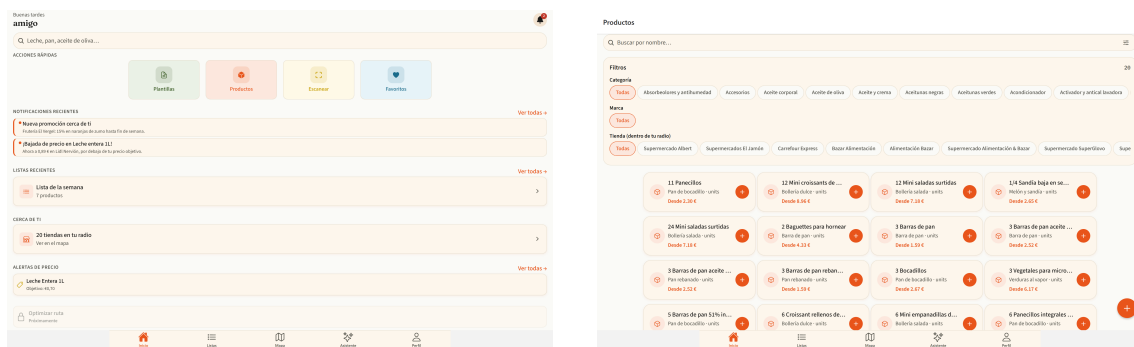


Figura 7.15: Versión de escritorio: pantalla principal con accesos horizontales (izquierda) y catálogo en rejilla multicolumna con soporte de teclado (derecha)

El resto de pantallas sigue el mismo patrón de adaptación; sus capturas se recogen en el Apéndice C para no sobrecargar este capítulo. En síntesis: la comparativa de precios (Figura C.1) gana una cabecera fija y exportación de la tabla a **CSV**; el detalle de lista añade el reordenado de ítems por arrastre y la exportación en texto plano; las colecciones de listas y plantillas (Figura C.2) se presentan como rejillas de tarjetas; el mapa ancla el panel de tiendas como columna lateral y el perfil de tienda adopta dos columnas (Figura C.3); las tiendas favoritas y el asistente conversacional (Figura C.4) centran su contenido a un ancho de lectura cómodo, con copia individual de cada respuesta del asistente; la ruta optimizada incorpora la exportación del itinerario y el módulo OCR delega en el selector de ficheros del sistema cuando no hay cámara (Figura C.5); y la bandeja de notificaciones y las alertas (Figura C.6) se centran a un ancho fijo.

Cada pantalla relevante de la versión web dispone además de URL propia, de modo que cualquier estado —una búsqueda concreta del catálogo mediante **?q=**, una lista, una tienda— puede guardarse como marcador o compartirse como enlace.

7.15– Portal Business para PYMEs

El portal *business* es la contrapartida del sistema para el pequeño comercio: una aplicación web independiente desde la que un establecimiento local publica sus precios y promociones para competir en igualdad de condiciones con las grandes cadenas dentro de BarGAIN. Su página de presentación (Figura 7.16) expone la propuesta de valor para el comercio —visibilidad ante consumidores cercanos, gestión autónoma de precios y promociones, y estimaciones orientativas de retorno— y resume el proceso de alta en tres pasos.



Figura 7.16: Página de presentación del portal Business, con la propuesta de valor para el comercio local y el proceso de alta en tres pasos

El acceso al portal (Figura 7.17) reutiliza el mismo sistema de autenticación JWT de la aplicación de consumo, con cuentas de rol *business*. Tras el registro, el comerciante completa el *onboarding* (Figura 7.18): un formulario con los datos legales y comerciales del negocio —razón social, CIF/NIF, dirección y sitio web— que crea el perfil de empresa en estado *pendiente*. Ningún comercio publica precios sin que un administrador haya verificado previamente esos datos, una salvaguarda imprescindible para sostener la confianza del consumidor en la información que muestra la aplicación.

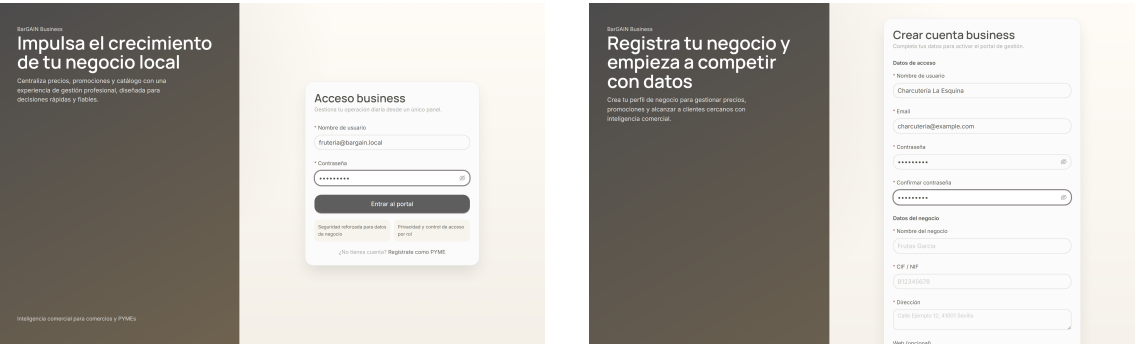


Figura 7.17: Acceso al portal Business: inicio de sesión (izquierda) y registro de una cuenta de comercio (derecha)



Figura 7.18: *Onboarding* del comercio: alta de los datos legales y comerciales que un administrador verificará antes de activar la cuenta

Una vez aprobado, el comerciante accede a su centro de operaciones (Figura 7.19): un panel ejecutivo con los indicadores esenciales del negocio en la plataforma —tiendas activas, precios publicados, promociones vigentes y visualizaciones de perfil acumuladas— , la salud de sus campañas y los últimos ajustes de precio aplicados. El propósito del panel es que un comerciante sin formación analítica pueda responder de un vistazo a las dos preguntas que le importan: qué estoy ofreciendo ahora mismo y cuánta atención está recibiendo.

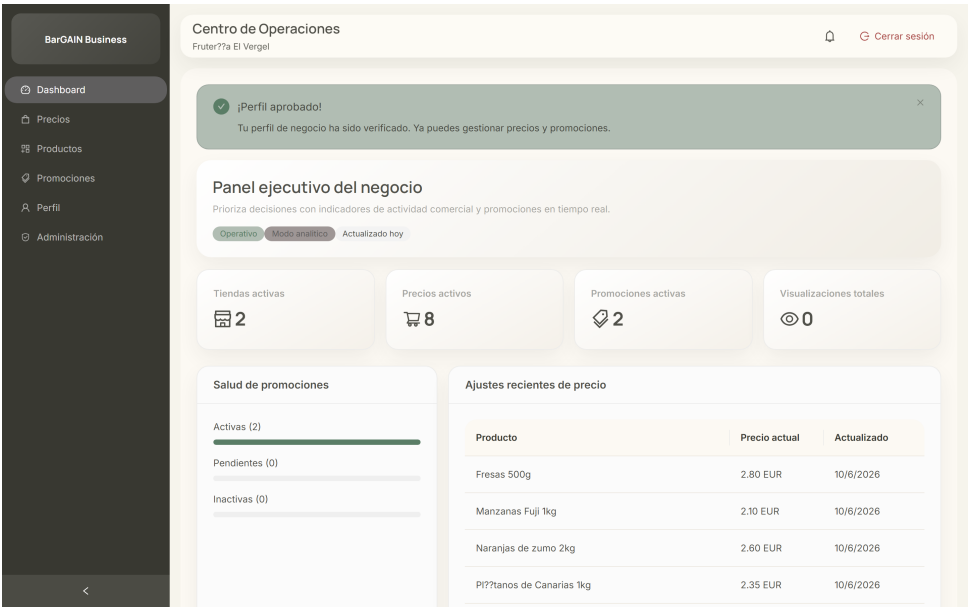


Figura 7.19: Centro de operaciones del comercio: indicadores de actividad, salud de promociones y ajustes recientes de precio

La gestión del surtido se apoya en dos vistas complementarias. La pantalla de productos (Figura 7.20, izquierda) resume el catálogo del comercio con el número de tiendas que ofrecen cada artículo, su precio base mínimo y la mejor oferta vigente. Para incorporaciones masivas, la carga por CSV (Figura 7.20, derecha) permite importar el catálogo completo en una sola operación, con validación fila a fila e informe de incidencias, lo que rebaja considerablemente la barrera de entrada para comercios con decenas de referencias.

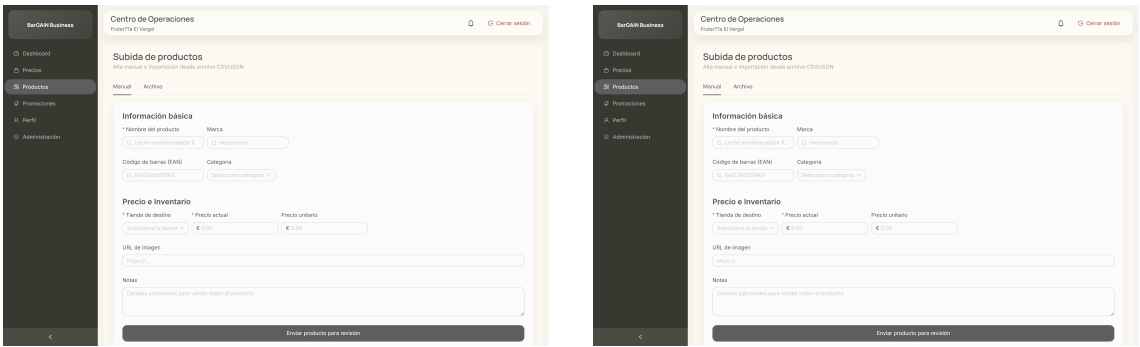


Figura 7.20: Gestión del surtido: resumen de productos del comercio (izquierda) y carga masiva de catálogo mediante CSV con validación (derecha)

El mantenimiento diario de precios se realiza en la tabla editable de la Figura 7.21: cada fila representa un precio publicado (producto–tienda) y el panel lateral permite modificar el precio base, el precio unitario y, opcionalmente, un precio de oferta con fecha de caducidad. Los cambios se publican de forma inmediata en la aplicación de consumo con la fuente *business*, que el sistema de prioridades de BarGAIN sitúa por delante de los datos de *scraping* por tratarse de información de primera mano.

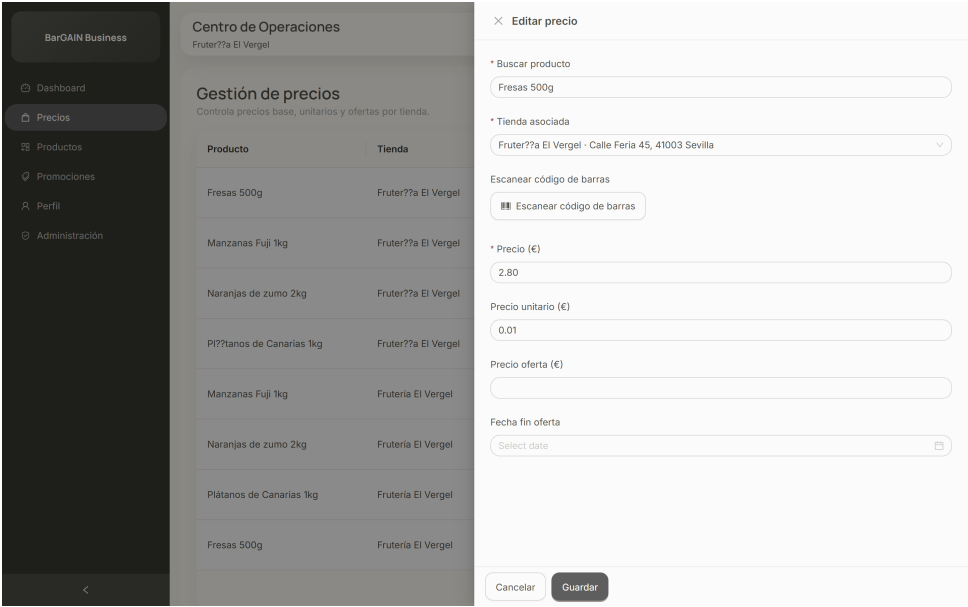


Figura 7.21: Gestión de precios: tabla editable con panel lateral de edición de precio base, unitario y oferta con caducidad

Sobre los precios se construyen las promociones (Figura 7.22, izquierda): descuentos fijos o porcentuales con fecha de inicio y fin, cantidad mínima opcional y título comercial, cuyo rendimiento (visualizaciones, estado) se supervisa desde el propio listado. Las promociones activas se difunden a los consumidores del entorno mediante notificaciones, cerrando el círculo entre la oferta del comercio y la demanda cercana. El perfil del negocio (Figura 7.22, derecha) completa la gestión con los datos públicos del establecimiento y el umbral de alerta competitiva: si un competidor del entorno publica un precio inferior en más del porcentaje configurado, el comerciante recibe un aviso automático.

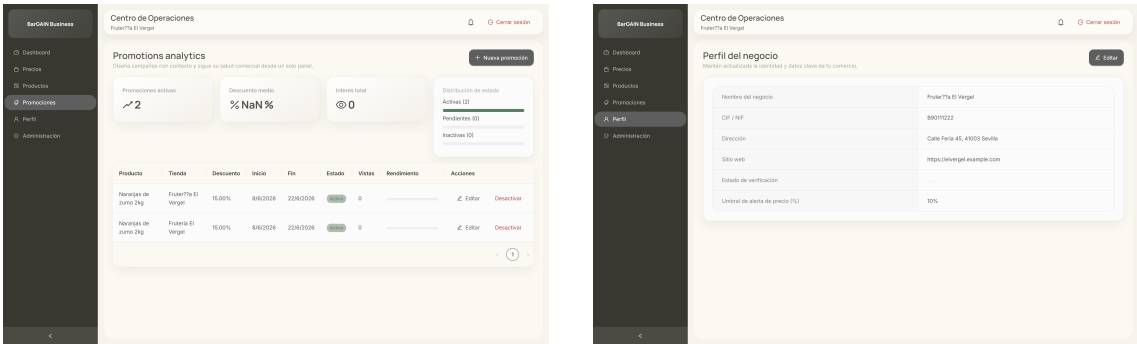


Figura 7.22: Promociones del comercio con analítica básica (izquierda) y perfil del negocio con umbral de alerta competitiva (derecha)

El ciclo administrativo se cierra en el panel de administración (Figura 7.23), reservado a usuarios con privilegios de gestión. En él se revisan los perfiles PYME pendientes —con sus datos legales a la vista— y las propuestas de producto remitidas por los consumidores, con acciones de aprobación y rechazo motivado. Este punto único de moderación garantiza que tanto los comercios como el catálogo crecen de forma controlada.

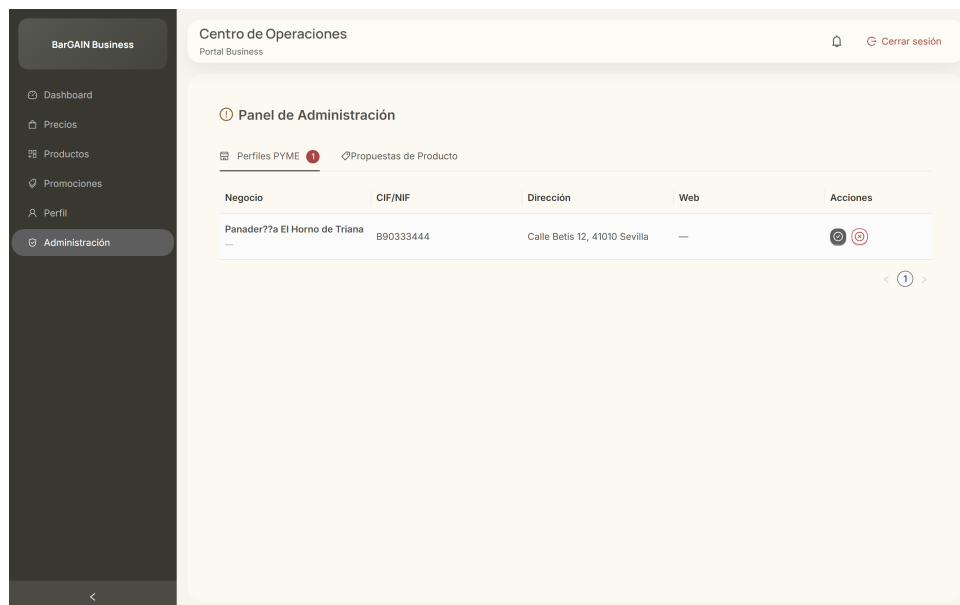


Figura 7.23: Panel de administración: verificación y aprobación de perfiles PYME y de propuestas de producto

7.16— Resolución de incidencias comunes

Se recogen, por último, las incidencias más habituales en el uso del sistema y su resolución:

- **El frontend no conecta con el backend:** verificar que el backend está levantado (`make dev`) y expuesto en `http://localhost:8000`.
- **Errores de autenticación al navegar:** cerrar sesión e iniciarla de nuevo para regenerar el par de tokens.
- **No aparecen tiendas en el mapa:** comprobar los permisos de ubicación del dispositivo o fijar unas coordenadas válidas desde las preferencias del perfil.
- **El OCR no reconoce texto:** procurar buena iluminación y texto legible; la opción de galería con una fotografía de mayor calidad suele resolver el problema.
- **La primera optimización tarda:** tras arrancar el backend, la primera optimización puede demorarse hasta 30 segundos (arranque en frío del servicio de rutas); las siguientes son prácticamente inmediatas.

Pruebas y Resultados

8.1— Objetivo

Validar que BarGAIN cumple los requisitos funcionales y no funcionales definidos en la Fase de Análisis, con cobertura automática de los flujos críticos y verificación documentada de los requisitos no funcionales clave (RNF-002, RNF-004, RNF-005).

8.2— Estrategia de testing

Se aplica una estrategia por capas, siguiendo el modelo de pirámide de tests:

- **Tests unitarios backend:** por módulo (`users`, `products`, `stores`, `prices`, `shopping_lists`, `business`, `notifications`, `optimizer`).
- **Tests de integración backend:** flujos cross-domain que ejercitan varios módulos en conjunto.
- **Tests de componentes frontend:** Jest + React Native Testing Library (111 tests en 15 suites), incluyendo los componentes y utilidades de la adaptación web.
- **Tests E2E con Playwright:** 4 flujos críticos del sistema integrado (backend + frontend web).
- **UAT manual en dispositivo móvil:** flujos nativos con GPS y cámara que no son automatizables mediante Playwright (verificados en iPhone con la app compilada).

8.3— Entorno de ejecución

- **Backend:** ejecución dentro del contenedor Docker (modelo híbrido ADR-002). Pytest con configuración en `backend/pytest.ini`, `DJANGO_SETTINGS_MODULE = config.settings.test`.
- **E2E Playwright:** ejecución nativa en host (no Docker) contra el backend Docker local o el staging de Render. Instalado en `frontend/web/` como dependencia de desarrollo.

Los comandos de ejecución principales son:


```
# Backend (desde raiz del repo)
make test-backend          # Tests con -v --tb=short
make test-backend-cov      # Con cobertura HTML

# E2E (desde host)
cd frontend/web && npx playwright test
cd frontend/web && npx playwright test --reporter=html
```

8.4– Suite de tests backend

8.4.1. Tests de integración

Los tests de integración verifican flujos API completos usando una base de datos de test real (sin mocks de BD), validando el comportamiento end-to-end del backend. La suite backend completa suma **333 tests**: 162 unitarios (17 archivos) y 171 de integración (17 archivos). El Cuadro 8.1 resume las suites de integración.

Cuadro 8.1: Suite de tests de integración backend

Archivo de test	Módulo	Cobertura funcional
test_auth_endpoints.py	users	Registro, login, refresh, perfil
test_optimizer_api.py	optimizer	POST /optimize/ con matriz mock
test_ocr_api.py	ocr	POST /ocr/scan/ con Vision API mock
test_business_verification.py	business	Aprobar/rechazar BusinessProfile
test_business_registration.py	business	Onboarding PYME
test_business_prices.py	business	Gestión de precios PYME
test_bulk_prices.py	business	Actualización masiva CSV
test_proposal_admin.py	business	Aprobar/rechazar propuestas
test_promotions.py	business	Gestión de promociones
test_list_endpoints.py	shopping_lists	CRUD de listas e ítems
test_cross_domain.py	core	Flujos cross-domain
test_notification_dispatch.py	notifications	Despacho push/email
test_notification_events.py	notifications	Eventos de notificación
test_price_endpoints.py	prices	API de precios
test_product_endpoints.py	products	API de productos
test_store_endpoints.py	stores	API de tiendas
test_assistant_api.py	assistant	Endpoint LLM con mock de Gemini

8.4.2. Cobertura

La suite backend mantiene una cobertura mínima del 80 % sobre los módulos de aplicación (apps/), medida con `pytest-cov`. El umbral no es declarativo sino **bloqueante**: el workflow `ci-backend.yml` ejecuta `pytest -cov=apps -cov-fail-under=80`, de modo que ninguna integración por debajo de ese umbral supera la CI; el informe se publica además en Codecov en cada build. La distribución de tests unitarios por módulo refleja dónde se concentra la lógica de negocio: `shopping_lists` (19), `products` (17), `prices` (15), `scraping` (22 entre spiders y pipeline), `stores` (11), `business` (11), `notifications` (11), `ocr` (9), `users` (8), `assistant` (8) y `optimizer` (15 entre solver, semántica y cliente ORS), además de tests de configuración (despliegue Render y comandos de seed).

8.5— Tests E2E con Playwright

Se desarrollaron 4 tests E2E automatizados con Playwright para verificar los flujos críticos del sistema integrado (backend + web companion Vite+React):

Cuadro 8.2: Suite de tests E2E con Playwright

Flujo	Archivo	Tipo	Descripción
Autenticación completa	<code>auth.spec.ts</code>	UI + API	Registro → login → refresh → logout
Lista → Optimizar → Ruta	<code>optimizer.spec.ts</code>	API-level	Crear lista → optimizar → verificar
OCR ticket → lista	<code>ocr.spec.ts</code>	API-level	Subir imagen → OCR → respuesta
Business portal	<code>business.spec.ts</code>	UI	Registro PYME → aprobación → panel

Los flujos de optimizador y OCR se implementan como tests API-level (usando el fixture `request` de Playwright, sin interfaz de usuario) porque estas funcionalidades son exclusivas de la app móvil — el web companion es un portal business y no dispone de pantallas de consumidor. Este enfoque verifica la integración backend end-to-end de los features principales del TFG sin requerir una interfaz web que no existe.

8.5.1. Configuración Playwright

- **Ubicación:** `frontend/web/playwright.config.ts`
- **Target:** web companion en `http://localhost:5173` (o `PLAYWRIGHT_BASE_URL`)
- **Backend:** Docker local o staging Render (`PLAYWRIGHT_API_URL`)
- **Navegador:** Chromium (un solo proyecto para evitar duplicación de estado de BD)
- **Workers:** 1 (secuencial — los tests comparten la misma base de datos)

8.6— UAT manual en dispositivo móvil

Los flujos que requieren APIs nativas del dispositivo (GPS, cámara) se verifican manualmente en un iPhone con la app compilada mediante el workflow `ios-build.yml`:

Cuadro 8.3: Resultados UAT en dispositivo móvil

Flujo UAT	Resultado
Solicitar ubicación y ver tiendas en mapa	Verificado
Crear lista y añadir ítems manualmente	Verificado
Capturar ticket con cámara y reconocer productos	Verificado
Lanzar optimización y navegar la ruta sugerida	Verificado
Chat con asistente LLM y verificar guardrail	Verificado

8.7— Validación de Requisitos No Funcionales

8.7.1. RNF-002: Disponibilidad del entorno de staging

El backend de staging se desplegó en Render.com (plan gratuito, blueprint `render.free.yaml`) con la siguiente arquitectura:

- Web Service único: Django con Gunicorn (2 workers) y un proceso Celery worker + beat embebido (`-concurrency 1`) para las tareas asíncronas.
- PostgreSQL 16 + PostGIS gestionado por la plataforma.
- Redis: broker Celery + caché Google Places.

El health check se configura en `GET /api/v1/health/` respondiendo HTTP 200. Los reinicios automáticos ante fallos se gestionan por la plataforma Render. Como optimización del arranque en frío del free tier, los ficheros estáticos se precompilan durante el build de la imagen Docker en lugar de generarse en el arranque del servicio.

Debe precisarse el alcance de la validación: el enunciado original de RNF-002 (disponibilidad mensual $\geq 99\%$) **no es medible tal cual en el free tier**, porque la plataforma hiberna el servicio tras unos minutos de inactividad (Sección 7.2) y no se ha desplegado monitorización continua de uptime. Lo que se ha verificado es la *disponibilidad funcional durante las ventanas de evaluación*: el health check responde correctamente tras cada despliegue, y los cuatro flujos E2E se ejecutan con éxito contra el staging público. La instrumentación de monitorización continua (y, con ella, la verificación del 99 % en sentido estricto) queda registrada como trabajo pendiente en el Capítulo 9.

8.7.2. RNF-004: Usabilidad y flujo máximo de 3 interacciones

Los flujos principales se completan en ≤ 3 interacciones en la app móvil:

- **Ver precios cercanos:** Home $\rightarrow$ Buscar $\rightarrow$ Resultados (3 taps)
- **Optimizar ruta:** Lista $\rightarrow$ Optimizar $\rightarrow$ Ver ruta (3 taps)
- **Añadir ítem OCR:** Lista $\rightarrow$ Cámara $\rightarrow$ Confirmar (3 taps)

Respecto al criterio de accesibilidad de RNF-004, la verificación realizada ha sido *heurística* (flujos cortos, etiquetas de formularios, estados de foco en la versión web y *tooltips* accesibles): no se ha ejecutado una auditoría formal WCAG 2.1 AA (World Wide Web Consortium (W3C), 2018) con herramientas específicas (contraste, navegación por teclado completa, lector de pantalla), que queda identificada como tarea previa a cualquier publicación en tiendas de aplicaciones.

8.7.3. RNF-005: Escalabilidad para 10.000 usuarios

La arquitectura de BarGAIN soporta el crecimiento a 10.000 usuarios concurrentes mediante cuatro decisiones de diseño complementarias:

1. **Workers Celery independientes:** el scraping, el OCR y la optimización se ejecutan en workers separados del API principal, evitando que tareas pesadas bloqueen la gestión de peticiones web.
2. **Redis como broker desacoplado:** las peticiones de usuario se encolan en Redis y se procesan de forma asíncrona, permitiendo absorber picos de carga sin timeout de petición.
3. **PostGIS con índices GiST:** las consultas de tiendas cercanas utilizan índices espaciales con complejidad $O(\log n)$, independiente del volumen total de tiendas registradas.
4. **API stateless con JWT:** el backend no mantiene estado de sesión en servidor. Escalar horizontalmente (añadir réplicas del Web Service) no requiere sincronización de sesiones.

8.8— Resumen de resultados

Cuadro 8.4: Resumen de resultados de la estrategia de testing

Tipo de test	Cantidad	Estado
Tests unitarios backend	162 (17 archivos)	Todos pasan
Tests de integración backend	171 (17 archivos)	Todos pasan
Tests frontend (Jest)	111 (15 suites)	Todos pasan
Tests E2E Playwright	4 flujos	Todos pasan
UAT manual móvil	5 flujos	Verificados
RNF-002 · Disponibilidad	Health check + E2E en staging	Verificación funcional
RNF-004 · Usabilidad	3 flujos	Verificación heurística (3 taps máx.)
RNF-005 · Escalabilidad	Justificación arquitectural	Documentado

8.9— Conclusión

La estrategia de testing multicapa garantiza la calidad del sistema a diferentes niveles de granularidad. La cobertura de tests backend se mantiene por encima del 80 % con umbral bloqueante en CI (333 tests entre unitarios y de integración), la suite frontend añade 111 tests de componentes y utilidades con Jest, y los 4 flujos E2E automatizados con Playwright verifican que el sistema integrado funciona correctamente de extremo a extremo contra el entorno de staging. Quedan explícitamente acotadas las validaciones pendientes: medición formal de latencias p95 (RNF-001), monitorización continua de disponibilidad (RNF-002), auditoría WCAG completa (RNF-004) y prueba de carga (RNF-005), todas recogidas como trabajo futuro en el Capítulo 9.

Conclusiones

9.1— Grado de cumplimiento

El proyecto BarGAIN ha completado todos los bloques de desarrollo planificados:

- **F1:** análisis, requisitos y diseño de arquitectura.
- **F2:** infraestructura base de desarrollo y CI/CD.
- **F3:** backend core completo con todos los módulos de dominio y API documentada.
- **F4:** frontend base y avanzado — autenticación, listas, catálogo, mapa, notificaciones, perfil y Places enrichment.
- **F5:** sistema de optimización multicriterio (OR-Tools + OpenRouteService), scraping de precios (Scrapy + Playwright, 11 spiders implementados y 4 en operación programada), OCR de tickets (Google Cloud Vision API + fuzzy matching), asistente LLM (Google Gemini 2.0 Flash Lite) y wiring de pantallas móviles a endpoints reales.
- **F6:** portal business completo para PYMEs — servicio de aprobación compartido, gestión de precios y promociones, tests de integración, UAT verificada, notificaciones push y email, sincronización documental.
- **F7:** pruebas integrales (unitarias, de integración y E2E), despliegue en staging y cierre documental de la memoria.

Adicionalmente, tras el cierre del plan inicial se incorporó una fase de adaptación de la app de usuario al uso web de escritorio (diseño responsive, atajos de teclado, exportaciones y deep-linking), documentada en los Capítulos 6 y 7.

El sistema está desplegado en staging (Render.com) y los cuatro flujos E2E críticos han sido verificados de forma automatizada con Playwright.

9.2— Aportaciones principales del trabajo

- **Optimizador multicriterio propio con capa semántica:** resolución semántico-difusa de cada ítem (trigramas + Gemini + fuzzy matching con desambiguación interactiva), consolidación de paradas y ordenación de la ruta con el routing solver de OR-Tools, con costes de arco ponderados (precio, distancia, tiempo) y distancias reales de conducción de OpenRouteService. Es el núcleo diferencial del TFG.

- **Pipeline de datos de precios automatizado:** once spiders Scrapy implementados y verificados con tests, de los que cuatro (Mercadona, Carrefour, Lidl y DIA) operan con programación periódica vía Celery Beat y normalización de producto contra catálogo interno.
- **OCR sobre tickets con Google Vision API:** extracción de texto, fuzzy matching contra catálogo y conversión automática a ítems de lista. Elimina la entrada manual.
- **Portal business para PYMEs:** flujo completo de registro, aprobación por administrador, gestión de precios y promociones, con visibilidad para el consumidor final.
- **Asistente LLM con guardrails de dominio:** Gemini 2.0 Flash Lite con restricción temática a consultas de compra, historial de conversación y rate limiting.
- **Arquitectura full-stack coherente:** separación clara entre apps Django, workers Celery independientes, PostGIS para geoespacial y React Native + Expo para iOS y web desde una única base de código.
- **Verificación automatizada:** suite de tests backend (unitarios + integración, >80 % cobertura), 111 tests de frontend con Jest, tests E2E con Playwright (4 flujos críticos) y UAT manual en iPhone.

9.3— Limitaciones

- **Cobertura de scraping:** el sistema opera de forma programada contra cuatro cadenas (Mercadona, Carrefour, Lidl, DIA). Los otros siete spiders implementados (Alcampo, Eroski, Hipercor, Consum, Covirán, Spar, Costco) están verificados a nivel de parser pero no validados de forma continuada contra los sitios reales, y las cadenas con protección agresiva contra scrapers requieren mantenimiento adicional. Asimismo, la opción global de respeto a `robots.txt` del proyecto Scrapy debe activarse antes de cualquier operación a escala (Capítulo 6).
- **Caducidad de precios:** los precios de scraping tienen TTL de 48 horas y los de crowdsourcing 24 horas; pueden no reflejar cambios de precio intradiarios.
- **Despliegue en plan gratuito (Render):** la infraestructura pública del proyecto se sostiene sobre el *free tier* de Render, lo que impone tres restricciones operativas. Primera, la *hibernación*: tras unos minutos sin tráfico la instancia se suspende y la siguiente petición sufre un *cold start* de entre 30 y 60 segundos mientras el contenedor se reactiva; el manual de usuario advierte de ello explícitamente. Segunda, los recursos computacionales (CPU y memoria compartidas) limitan el rendimiento del optimizador en escenarios de carga. Tercera, la base de datos del plan gratuito presenta restricciones de capacidad y vigencia que la hacen apta para demostración, pero no para producción. La migración a un plan de pago o a la infraestructura AWS prevista en el Capítulo 6 eliminaría estas tres limitaciones sin cambios de código.
- **Certificado iOS gratuito:** la distribución con Sideloadly y Apple ID gratuito genera certificados con caducidad de 7 días, que requieren re-sideload periódico para el dispositivo de demo.
- **RNF-005 sin load test real:** la justificación de escalabilidad a 10.000 usuarios concurrentes es arquitectural (Celery workers independientes, PostGIS spatial index, API stateless). Un load test completo requeriría infraestructura de pago fuera del alcance del TFG.

- **Validaciones no funcionales pendientes:** no se ha realizado una campaña formal de medición de latencias p95 (RNF-001) ni se ha instrumentado monitorización continua de disponibilidad (RNF-002); la conformidad WCAG 2.1 AA (RNF-004) se ha verificado solo de forma heurística. Las tres validaciones quedan acotadas en el Capítulo 8 y recogidas como trabajo futuro.
- **Una única ruta recomendada:** el diseño preliminar contemplaba devolver tres rutas alternativas; la implementación final prioriza una única ruta con desambiguación semántica interactiva (Capítulo 6). La generación de alternativas se pospone como trabajo futuro.

9.4— Lecciones aprendidas

- **Modelo híbrido backend Docker / frontend nativo en host:** determinante para la productividad en Windows. Docker volumes en Windows rompen el HMR de Metro/Expo; ejecutar el frontend nativo elimina este problema manteniendo el backend aislado.
- **Contratos API explícitos y documentación viva:** mantener TASKS.md, ADRs y la memoria actualizados durante el desarrollo (no al final) reduce la fricción entre backend y frontend y facilita la incorporación de agentes IA al flujo.
- **OR-Tools frente a algoritmo propio:** usar la librería de optimización combinatoria de Google (producción en empresas logísticas) es preferible a reinventar el VRP desde cero. El valor del TFG está en la integración y la función de scoring, no en reimplementar un solucionador genérico.
- **OCR cloud vs local:** Google Vision API supera a Tesseract en precisión sobre tickets de supermercado reales (texto impreso variable), justificando el coste de la llamada API frente a la alternativa local.
- **Playwright request fixture para flujos mobile-only:** los flujos de lista y optimizador son exclusivos de la app móvil. Cubrirlos con tests API-level (Playwright request fixture) permite automatización completa sin necesitar una interfaz web consumer.

9.5— Trabajo futuro

1. **Publicación en App Store y Google Play:** el proyecto está técnicamente listo para iniciar el proceso de publicación. Requiere cuenta de desarrollador Apple/-Google y revisión de las guías de contenido.
2. **Activación y ampliación de scrapers:** poner en operación programada y validar contra los sitios reales los siete spiders ya implementados (Alcampo, Eroski, Hipercor, Consum, Covirán, Spar, Costco), activar el respeto global a `robots.txt`, e incorporar nuevas cadenas (Aldi, El Corte Inglés alimentación) y PYMEs locales vía integración API propia.
3. **Precios en tiempo real:** varios supermercados están desplegando APIs semi-públicas o feeds RSS de precios; la arquitectura está preparada para incorporarlos como fuente `api` en el modelo `Price`.

4. **Sistema de reseñas de productos:** los usuarios podrían valorar la calidad/frescura de productos por tienda, añadiendo una dimensión cualitativa al comparador.
5. **Suscripción premium para PYMEs:** el modelo de negocio contempla tres tiers (**free**, **basic**, **premium**) con funcionalidades diferenciadas; la implementación actual es el tier gratuito.
6. **Internacionalización:** la arquitectura soporta múltiples países; la expansión requiere scrapers por mercado y ajuste de la lógica de normalización de cadenas.
7. **Rutas alternativas y validación no funcional completa:** generar las top-3 rutas alternativas del diseño preliminar sobre el pipeline actual, instrumentar monitorización continua de disponibilidad, medir latencias p95 contra RNF-001 y ejecutar una auditoría WCAG 2.1 AA (World Wide Web Consortium (W3C), 2018) con prueba de carga para RNF-005.

9.6— Cierre

BarGAIN demuestra que es posible construir, en el marco de un TFG, un sistema full-stack de complejidad real que integra optimización combinatoria, geoespacial, procesamiento de imágenes, generación de lenguaje natural y scraping automatizado bajo una arquitectura coherente y verificada. El proyecto cumple los objetivos planteados en la fase de análisis —con las desviaciones de diseño y validaciones pendientes documentadas explícitamente en los Capítulos 6 y 8— y deja una base técnica sólida para su evolución como producto real.

Catálogo de requisitos

Este apéndice recoge el catálogo completo de requisitos del sistema. La especificación extendida, con criterios de aceptación e historias de usuario (HU-001 a HU-030), se mantiene versionada en el repositorio (`docs/memoria/07-requisitos.md`).

A.1– Requisitos de información

RI-001 Usuarios. Identidad (nombre de usuario único, email único, contraseña hashada), rol (consumidor, comercio, administrador), ubicación por defecto (PointField SRID 4326), radio de búsqueda, máximo de paradas, pesos de optimización y preferencias de notificación.

RI-002 Productos. Catálogo normalizado: nombre, nombre normalizado, EAN-13 opcional, categoría, marca, unidad, imagen y estado.

RI-003 Categorías. Jerarquía navegable con categoría padre y orden de visualización.

RI-004 Marcas. Nombre, logotipo y vínculo opcional con cadena (marca blanca).

RI-005 Cadenas. Nombre, logotipo, web, disponibilidad de API oficial y color corporativo.

RI-006 Tiendas. Establecimiento físico geolocalizado (PointField con índice GiST), cadena opcional, horario JSON, indicador de comercio local y nivel de suscripción.

RI-007 Precios. Precio actual e histórico por par producto–tienda: precio, precio unitario, oferta y caducidad, fuente (API/scraping/comercio/crowdsourcing), verificación y bandera de caducidad.

RI-008 Listas. Listas por usuario con ítems de texto libre normalizado, cantidad y estado de comprado; la vinculación a productos se resuelve de forma diferida.

RI-009 Optimizaciones. Resultado persistente: ubicación, radio, pesos, totales, ruta (paradas ordenadas e ítems por parada) y preferencias semánticas confirmadas.

RI-010 Perfiles de negocio. Datos fiscales (NIF/CIF único), estado de verificación, umbral de alerta competitiva y promociones asociadas.

RI-011 Sesiones OCR. Imagen procesada, texto extraído, candidatos reconocidos con confianza y estado de la sesión.

RI-012 Conversaciones. Historial de mensajes usuario/asistente con fechas y estado.

A.2– Requisitos funcionales

Cuadro A.1: Catálogo de requisitos funcionales

ID	Nombre	Síntesis
RF-001	Registro de usuario	Alta con nombre de usuario, email único y contraseña validada.
RF-002	Inicio de sesión	Autenticación con nombre de usuario y contraseña; par de tokens JWT con rotación; tiempos de vida configurables por entorno (60 min/7 días en la configuración de referencia).
RF-003	Gestión de perfil	Consulta y modificación de datos personales y preferencias.
RF-004	Preferencias de optimización	Pesos relativos de precio/distancia/tiempo, radio (1–25 km) y paradas (1–4).
RF-005	Recuperación de contraseña	Restablecimiento por enlace de correo con caducidad.
RF-006	Consulta de productos	Búsqueda con filtros, paginación y ordenación.
RF-007	Detalle de producto	Ficha completa con rango de precios en tiendas cercanas.
RF-008	Autocompletado	Sugerencias en tiempo real con matching fuzzy sobre nombre normalizado.
RF-009	Categorías	Jerarquía de categorías navegable.
RF-010	Alta por crowdsourcing	Propuesta de producto sujeta a aprobación del administrador.
RF-011	Tiendas por proximidad	Listado por radio configurable ordenado por distancia.
RF-012	Detalle de tienda	Información completa y precios de la lista activa del usuario.
RF-013	Mapa	Mapa interactivo con marcadores diferenciados por cadena/comercio local.
RF-014	Tiendas favoritas	Marcado de favoritas con acceso rápido y notificaciones.
RF-015	Comparación por producto	Precios en todas las tiendas del radio con fuente y antigüedad.
RF-016	Comparación de cesta	Coste total de la lista en cada tienda individual.
RF-017	Histórico de precios	Evolución temporal del precio con gráfico.
RF-018	Alerta de precio	Notificación al alcanzar un precio objetivo.
RF-019	Crowdsourcing de precios	Reporte de precios con caducidad de 24 h.
RF-020	Gestión de listas	CRUD de listas con límite de 20 activas.
RF-021	Gestión de ítems	Texto libre normalizado, cantidades y marcado de comprado.
RF-022	Compartir lista	Edición colaborativa con otros usuarios registrados.
RF-023	Plantillas	Listas reutilizables instanciadas.
RF-024	Optimización de ruta	Asignación ítem→(producto, tienda) con resolución semántico-difusa y ruta recomendada ordenada con OR-Tools según pesos del usuario; ambigüedades resolubles con recálculo.
RF-025	Ruta en mapa	Recorrido, paradas y productos por parada sobre mapa interactivo.
RF-026	Desglose de ahorro	Precio total, distancia, tiempo y desglose por parada.
RF-027	Recálculo bajo demanda	Nueva optimización en menos de 5 s al cambiar parámetros.
RF-028	Captura OCR	Foto de lista o ticket procesada con OCR.

ID	Nombre	Síntesis
RF-029	Matching OCR	Candidatos del catálogo con nivel de confianza y corrección manual.
RF-030	Consulta en lenguaje natural	Chat con respuestas contextualizadas con datos reales.
RF-031	Sugerencias de ahorro	Estrategias de ahorro limitadas al dominio de compra.
RF-032	Registro de negocio	Alta PYME con datos fiscales sujeta a verificación.
RF-033	Precios del comercio	Actualización manual con fuente verificada sin caducidad.
RF-034	Promociones	Creación y gestión de promociones con vigencia.
RF-035	Notificaciones	Push y email por eventos según preferencias.

A.3— Requisitos no funcionales

RNF-001 Rendimiento. API con p95 inferior a 500 ms en operaciones CRUD e inferior a 5 segundos para la optimización de rutas.

RNF-002 Disponibilidad. Disponibilidad mínima del 99 % mensual en staging, excluyendo mantenimientos. (Véase el alcance real de su validación en el Capítulo 8.)

RNF-003 Seguridad. HTTPS obligatorio, JWT con rotación, rate limiting (100/h anónimos, 1.000/h autenticados), hashing seguro de contraseñas y cifrado de datos sensibles.

RNF-004 Usabilidad. Directrices WCAG 2.1 AA (World Wide Web Consortium (W3C), 2018), diseño responsive y acciones principales en un máximo de 3 interacciones.

RNF-005 Escalabilidad. Crecimiento hasta 10.000 usuarios concurrentes mediante workers Celery independientes y API stateless.

RNF-006 Mantenibilidad. Cobertura de tests ≥ 80 % verificada de forma bloqueante en CI; convenciones PEP 8 y ESLint/Prettier.

RNF-007 Compatibilidad. iOS 15+, Android 10+ y las dos últimas versiones de los navegadores principales.

RNF-008 Protección de datos. Cumplimiento del RGPD (Parlamento Europeo y Consejo de la Unión Europea, 2016): consentimiento explícito de ubicación, derecho al olvido y minimización de datos.

A.4— Reglas de negocio

RN-001 Radio de búsqueda entre 1 y 25 km (10 km por defecto).

RN-002 Máximo 4 paradas por ruta (3 por defecto).

RN-003 Caducidad de precios: 48 h scraping, 24 h crowdsourcing; sin caducidad para comercio verificado.

RN-004 Prioridad de fuentes: API oficial > scraping > comercio verificado > crowdsourcing.

RN-005 Rechazo de precios no positivos y marcado de precios anómalos (>200 % de la media) como sospechosos.

RN-006 Máximo 20 listas activas por usuario.

RN-007 El asistente solo responde consultas del dominio de compra.

RN-008 Verificación administrativa previa de perfiles de comercio.

RN-009 Respeto a las directivas `robots.txt` (Koster y cols., 2022) y exclusión de cadenas que prohíban el acceso automatizado.

RN-010 Frecuencia máxima de scraping de una vez cada 24 h por cadena, con retardo mínimo de 2 segundos entre peticiones.

A.5– Trazabilidad requisitos–módulos–pruebas

Cuadro A.2: Trazabilidad de dominios funcionales a módulos y suites de prueba

Dominio (RF)	Módulo	Suites de prueba
Autenticación (001–005)	apps/users	test_users, test_auth_endpoints, auth.spec
Catálogo (006–010)	apps/products	test_products, test_product_endpoints
Tiendas (011–014)	apps/stores	test_stores, test_store_endpoints
Precios (015–019)	apps/prices	test_prices, test_price_endpoints
Listas (020–023)	apps/shopping_lists	test_shopping_lists, test_list_endpoints
Optimizador (024–027)	apps/optimizer	test_optimizer*, test_distance_ors, optimizer.spec
OCR (028–029)	apps/ocr	test_ocr, test_ocr_api, ocr.spec
Asistente (030–031)	apps/assistant	test_assistant, test_assistant_api
Business (032–034)	apps/business	test_business_*, business.spec
Notificaciones (035)	apps/notifications	test_notification_*

APÉNDICE B

Relación de endpoints de la API

Todos los endpoints cuelgan del prefijo `/api/v1/` y, salvo indicación contraria, requieren autenticación JWT. La especificación OpenAPI completa se publica en `/api/v1/schema/` (con interfaces Swagger UI y ReDoc).

Cuadro B.1: Endpoints de la API por módulo

Método	Ruta	Descripción
GET	<code>health/</code>	Health check público (exento de throttling)
POST	<code>auth/register/</code>	Registro de usuario (público)
POST	<code>auth/token/</code>	Obtención de tokens JWT (público)
POST	<code>auth/token/refresh/</code>	Renovación del token de acceso
POST	<code>auth/password-reset/</code>	Solicitud de restablecimiento (público)
POST	<code>auth/password-reset/confirm/</code>	Confirmación de restablecimiento
GET/PATCH	<code>auth/profile/</code>	Perfil y preferencias del usuario
GET	<code>products/</code>	Listado y búsqueda fuzzy de productos
GET	<code>products/{id}/</code>	Detalle de producto
GET	<code>products/categories/</code>	Jerarquía de categorías
POST	<code>products/proposals/</code>	Propuesta de producto (crowdsourcing)
*	<code>products/proposals/admin/</code>	Moderación de propuestas (administrador)
GET	<code>stores/</code>	Búsqueda geoespacial (<code>lat</code> , <code>lng</code> , <code>radius_km</code>) o favoritas
GET	<code>stores/{id}/</code>	Detalle de tienda
POST	<code>stores/{id}/favorite/</code>	Alternar tienda favorita
POST	<code>prices/compare/</code>	Comparativa de un producto entre tiendas
POST	<code>prices/list-total/</code>	Coste total de una lista por tienda
POST	<code>prices/crowdsource/</code>	Aporte de precio por crowdsourcing
GET	<code>prices/{product_id}/history/</code>	Histórico de precios
*	<code>prices/alerts/</code>	CRUD de alertas de precio
*	<code>lists/</code>	CRUD de listas de la compra e ítems
*	<code>lists/templates/</code>	CRUD de plantillas de lista
POST	<code>optimize/</code>	Calcular o recalcular la ruta óptima
GET	<code>optimize/?shopping_list_id={id}</code>	Última ruta persistida de una lista
POST	<code>optimize/choices/</code>	Resolución de ambigüedad semántica y recálculo

Método	Ruta	Descripción
POST	ocr/scan/	Procesado OCR de imagen (límite 60/h)
POST	assistant/chat/	Mensaje al asistente LLM (límite 30/h)
*	business/profiles/	Alta y gestión de perfiles PYME, con acciones de aprobación y rechazo para administradores
*	business/prices/	Gestión de precios del comercio (incl. carga CSV)
*	business/promotions/	Gestión de promociones
*	notifications/	Buzón de notificaciones (lectura, borrado lógico)
POST	notifications/push-token/	Registro del token push del dispositivo

Las filas marcadas con * corresponden a *routers* DRF que exponen el conjunto estándar de operaciones (GET de colección y detalle, POST, PATCH, DELETE) más las acciones específicas indicadas.

APÉNDICE C

Capturas de la versión de escritorio

Este apéndice complementa la Sección 7.14 con las capturas restantes de la aplicación de usuario ejecutada en navegador de escritorio (ventana de 1440×900).

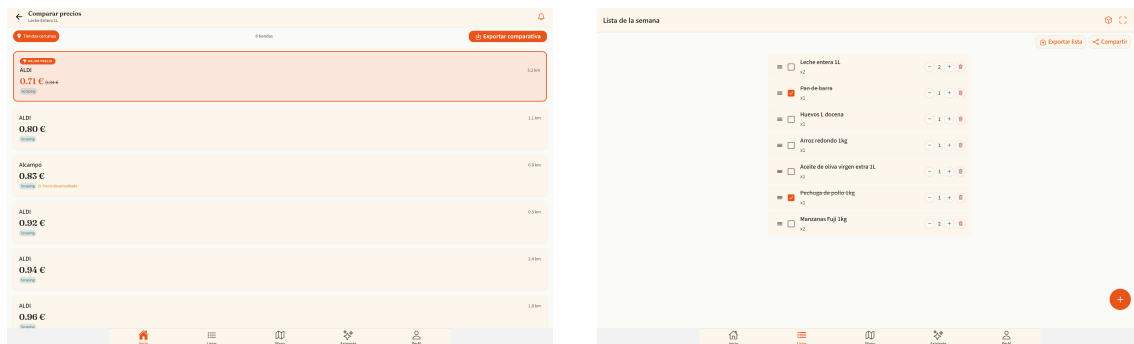


Figura C.1: Comparativa de precios con cabecera fija y exportación CSV (izquierda) y detalle de lista con reordenado por arrastre y exportación (derecha)

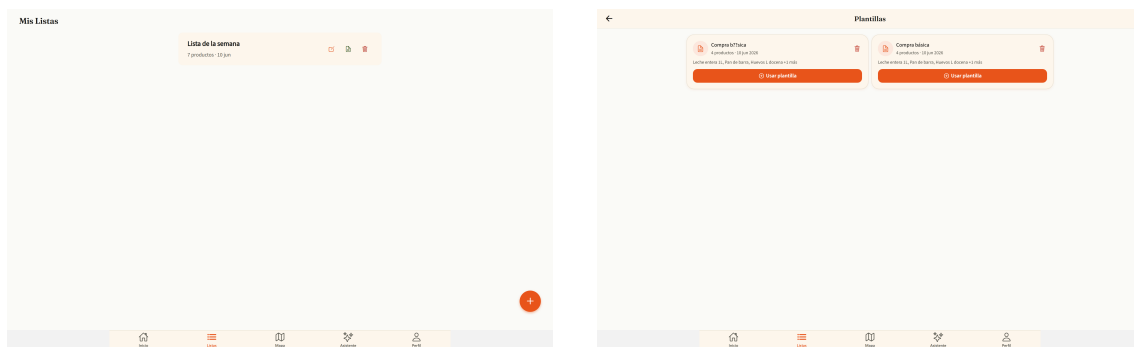


Figura C.2: Colecciones en rejilla responsiva: listas de la compra (izquierda) y plantillas (derecha)

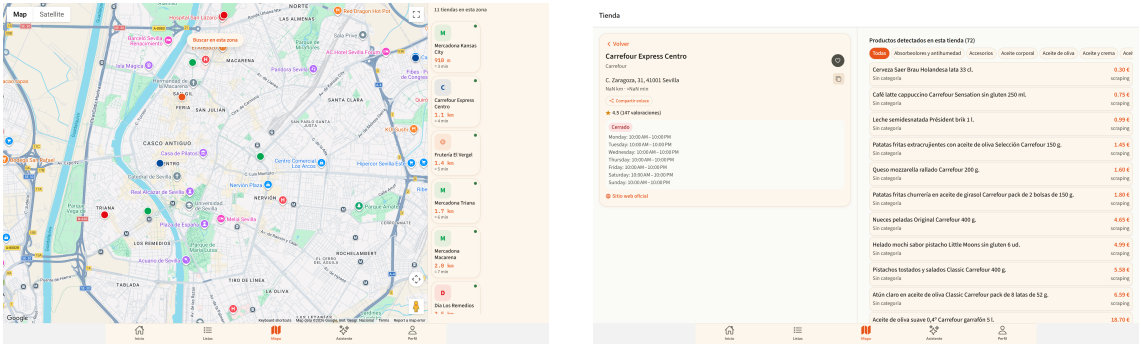


Figura C.3: Mapa con panel lateral de tiendas (izquierda) y perfil de tienda a dos columnas (derecha)

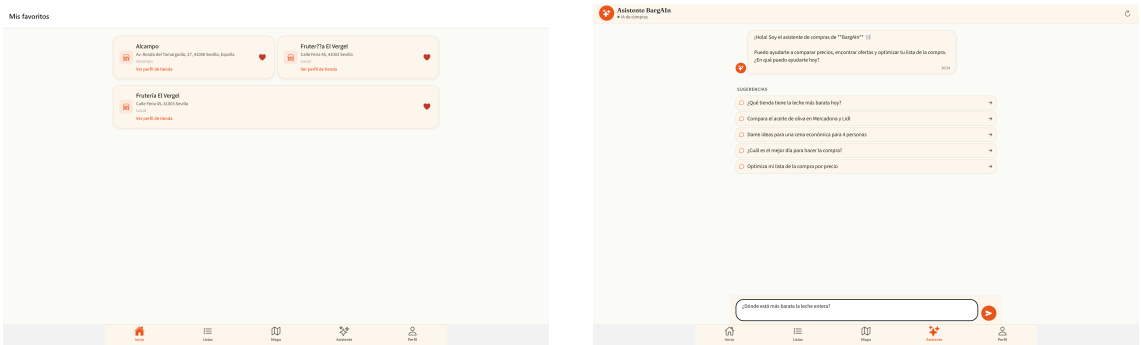


Figura C.4: Tiendas favoritas en rejilla (izquierda) y asistente conversacional centrado con copia por mensaje (derecha)

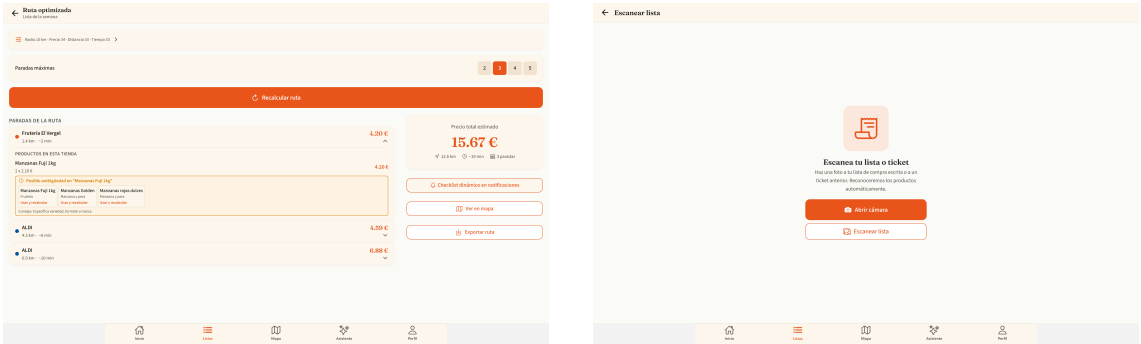


Figura C.5: Ruta optimizada con exportación del itinerario (izquierda) y módulo OCR adaptado al escritorio (derecha)

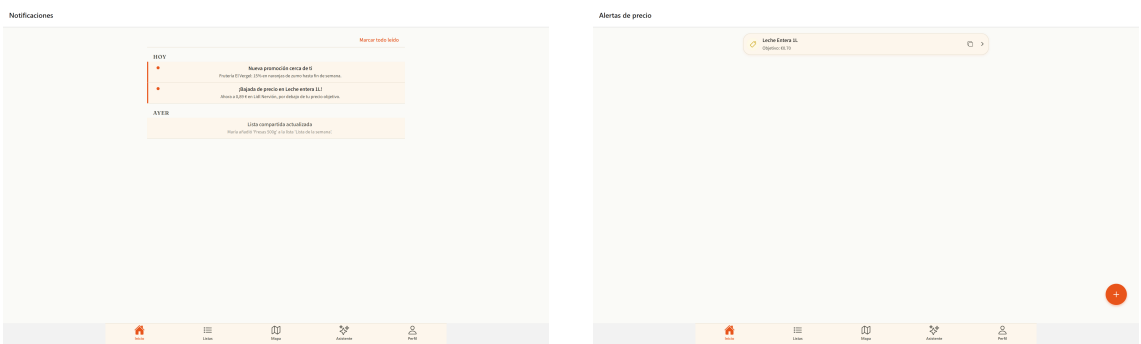


Figura C.6: Bandeja de notificaciones (izquierda) y alertas de precio (derecha) en la versión de escritorio

Referencias

- Applegate, D. L., Bixby, R. E., Chvatal, V., y Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- Brown, T. B., y cols. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>. (Accessed: 2026-04-09)
- Encode OSS Ltd. (2026). *Django rest framework documentation*. <https://www.django-rest-framework.org/>. (Accessed: 2026-04-09)
- Google. (2026). *Gemini api documentation*. <https://ai.google.dev/gemini-api/docs>. (Accessed: 2026-06-12)
- Google Cloud. (2026). *Cloud vision documentation*. <https://cloud.google.com/vision/docs>. (Accessed: 2026-04-09)
- Google OR-Tools Team. (2026). *Or-tools documentation*. <https://developers.google.com/optimization>. (Accessed: 2026-04-09)
- Heidelberg Institute for Geoinformation Technology (HeiGIT). (2026). *Openrouteservice documentation*. <https://openrouteservice.org/>. (Accessed: 2026-06-12)
- Instituto Nacional de Estadística. (2026). *Indice de precios de consumo (ipc). base 2021*. https://www.ine.es/dyngs/INEbase/es/operacion.htm?c=Estadistica_C&cid=1254736176802. (Accessed: 2026-06-12)
- Jazzband. (2026). *Simple jwt: A json web token authentication plugin for django rest framework*. <https://django-rest-framework-simplejwt.readthedocs.io/>. (Accessed: 2026-06-12)
- Jefatura del Estado. (2002). *Ley 34/2002, de 11 de julio, de servicios de la sociedad de la informacion y de comercio electronico*. Boletín Oficial del Estado, BOE-A-2002-13758.
- Junta de Andalucía. (2018). *Informe final de la consulta preliminar del mercado de perfiles profesionales en el ambito informatico*.
- Koster, M., Illyes, G., Zeller, H., y Sassman, L. (2022). *Rfc 9309: Robots exclusion protocol*. <https://www.rfc-editor.org/rfc/rfc9309>. (Internet Engineering Task Force (IETF))
- Lewis, P., y cols. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Luxen, D., y Vetter, C. (2011). Real-time routing with openstreetmap data. En *Proceedings of the 19th acm sigspatial international conference on advances in geographic information systems*. doi: 10.1145/2093973.2094062
- Microsoft Playwright Team. (2026). *Playwright documentation*. <https://playwright.dev/docs/intro>. (Accessed: 2026-04-09)
- Obe, R. O., y Hsu, L. S. (2021). *Postgis in action* (3rd ed.). Manning Publications.
- Organizacion de Consumidores y Usuarios (OCU). (2026). *Ocu market*. <https://www.ocu.org/>. (Accessed: 2026-03-11)

- Parlamento Europeo y Consejo de la Union Europea. (2016). *Reglamento (ue) 2016/679 relativo a la proteccion de las personas fisicas en lo que respecta al tratamiento de datos personales (rgpd)*. Diario Oficial de la Union Europea, L 119.
- PostGIS Project Steering Committee. (2026). *Postgis documentation*. <https://postgis.net/documentation/>. (Accessed: 2026-04-09)
- PostgreSQL Global Development Group. (2026). *Postgresql 16 documentation*. <https://www.postgresql.org/docs/16/>. (Accessed: 2026-04-09)
- Scrapy Developers. (2026). *Scrapy documentation*. <https://docs.scrapy.org/>. (Accessed: 2026-04-09)
- SeatGeek. (2026). *Thefuzz: Fuzzy string matching in python*. <https://github.com/seatgeek/thefuzz>. (Accessed: 2026-06-12)
- Soysuper S.L. (2026). *Soysuper: tu supermercado de supermercados*. <https://soysuper.com/>. (Accessed: 2026-03-11)
- World Wide Web Consortium (W3C). (2018). *Web content accessibility guidelines (wcag) 2.1. w3c recommendation*. <https://www.w3.org/TR/WCAG21/>. (Accessed: 2026-06-12)